Users' Guide to Python CodeBase Tools

by James Heuer, President M-P Systems Services, Inc.

Portland, Oregon, USA

July, 2026, Version 1.02

Contents

Introduction

What is Python CodeBase Tools?

Python CodeBase Tools is a sophisticated wrapper around the C Language DLL called CodeBase, originally developed by Sequiter Software Inc., which is now available as an open source module on GitHub. Simply put, CodeBase allows programmatic access to the power of data persistence using the DBF data table format. This tool can be applied to many applications, from analysis utilities to full-blown multi-module applications which must share complex, multi-table data in real-time. At the present time, the underlying DLL components were compiled for Microsoft Windows and are only available for that platform.

In its raw form, accessible directly by C programs and other applications that can import shared library functions from a Windows C .DLL file, CodeBase provides an extremely granular tool-set with hundreds of functions which must be used in concert to accomplish basic "CRUD" (Create, Retrieve, Update, and Delete) record handling and table creation.

In contrast, Python CodeBase Tools encapsulates CodeBase in a module compiled with the Python C API to simplify the programming interface while taking advantage of the inherent performance of the Python C API. The result is an importable module usable by any Windows Python script in either 32 or 64-bit versions from Python 3.10 and up (32-bit Python versions 3.6 to 3.9 are also supported). Your program can simply instantiate the cbTools object class, and call powerful functions directly in your Python script to manipulate and massage your DBF-based data.

Why Use the Python CodeBase Tools?

Conventional wisdom tells programmers to use one of the SQL-based components available to Python programmers, possibly using the more-or-less generic SqlAlchemy as a wrapper. These components generally require some degree of facility with SQL language and queries. They also isolate you from your data – often masking where and how it is stored and which physical tables (if even contained in separate files, which isn't assured) actually contain the data you are handling. And while these systems may offer great sophistication, their complexity is also often overkill for everyday requirements. Conversely, simpler versions like SQLite have limited functionality in terms of field data types, multi-user access, speed, index construction, record level locking and other useful features which are built in with CodeBase and the DBF table format.

Python CodeBase Tools uses the underlying CodeBase C DLL technology directly from Python, providing a highly efficient, performant, easy to program, and reliable platform to build analytical tools and medium sized multi-user applications. All the while it insulates the user from the confusing granularity of the underlying CodeBase components.

Python CodeBase Tools (and commercial products like Visual FoxPro which inspired it) supports a "navigational" approach to data management rather than the convoluted SQL relational approach, which puts abstruse relational logic between you and your data. This means that with CodeBase Tools once a table is opened, the programmer can traverse the table forward and backward with simple navigational commands, inspect a "current" record, locate a record very quickly by a primary key, by a complex specialized index key, or by unique record number, and otherwise have immediate and direct access to the records in the table for both reading and updating. We provide a more complete explanation of how Navigational Table Access provides programmers powerful and easy-to-use tools to solve many data persistence problems.

Purpose of this Document

Though Python CodeBase Tools simplifies the underlying CodeBase access to DBF tables, it still offers the user some 100 functions for manipulating and managing data. But don't feel overwhelmed… it is possible to create and populate data tables using just a few of these functions. This document is intended to get you moving quickly to exploit the power of this set of tools.

Brief Background: Navigational Table Access versus SQL Access

A Different Paradigm

CodeBase Tools exploits the native "Navigational Table Access" to your data tables that made xBase components and their DBF tables so fast and easy to manage. The basic idea as implemented in CodeBase Tools is that navigational access treats a database (DBF) table as a physical sequence of records that you move through one at a time, like advancing through a tape forward or backward. Rather than describing what data you want (as SQL does), you tell the system where to go — move to the first record, skip forward ten, seek to a key value, read what's there. The "current record", is the central abstraction: it is the record currently in the systems record buffer, and all read/write operations related to the "current record" act on that record.

This stands in sharp contrast to the set-based model of SQL, where a query returns a whole result set and the engine decides how to fetch it. In navigational systems, the programmer controls traversal explicitly and completely.

Similarly, there is the concept of the "currently selected table". When you first open the DBF table in CodeBase Tools, that table becomes the currently selected table, even if you previously opened other tables. When that table is opened, the system assigns it an "alias name", which allows you to make a direct reference to that table later in your code and to access the "current record" in that table. Each table currently open may have its own current record. When you "select" an existing alias to make its table the currently selected table, then your read/write operations and navigation commands work on that table and set or move its current record.

The ”current record” may have one of three states:

Is a valid record — a specific row number (1-based) in the table is the current record.

BOF (Beginning of File) — There is no current record, and the “record pointer” is logically set to the top of the file before the first physical record. The bof() function returns True.

EOF (End of File) — There is no current record, and the “record pointer” has been moved past the last record in the table, so that the eof() function returns True.

Reading a field value always means reading from the current record. Writing means overwriting the current record in place. There is no implicit "fetch next" — you move the pointer yourself, then act. Attempting to read from or write to a record when the table state is BOF or EOF results in an error condition.

There are functions in CodeBase Tools which move the record pointer (current record) in the currently selected table. These correspond to similar commands in VFP or xBase languages:

- goto(gomode=””, gonum=1) – The most direct instruction function for moving the current record. The allowable values for gomode are:
    - “RECORD” – pass a record number to the gonum parameter, and the current record pointer will be moved to that specific record number if it exists.
    - “NEXT” – moves to the next record in order, if present. The gonum parm is ignored. Same as goto(“SKIP”, 1).
    - “PREV” – moves backward to the previous record, if present. The gonum parm is ignored. Same as goto(“SKIP”, -1)
    - “SKIP” – moves forward in the table by the number of records indicated by the gonum parameter, forward (positive values) or backward (negative values)
- The functions seek() and locate() can also be used to move to a specific record. These are further explained in subsequent sections of this document.

This is explicit, imperative, and completely transparent — every record visit is a deliberate programmer action. But to this point, you are only traversing the table in its original record number (physical) order determined by the sequence in which they were appended to the table. But there’s more…

The Role of Index Files

Without an index, the record pointer moves through records in physical order — the order they were appended to the .DBF file. Deleted records leave gaps (marked with a deletion flag but not immediately removed – see below), and the sequence reflects insertion history rather than any meaningful sort.

Index files change this. The default implementation of CodeBase Tools makes use of a high performance .CDX type index file which can be associated with each DBF table. This may contain a large number of individual index "tags", each defining a specific ordering. When an index is set active via setorderto(), navigation still moves the pointer one record at a time, but the logical sequence follows the index key order rather than physical order. The goto() family of function instructions follow the index ordering rather than the raw record number order in this instance. Indexes may be defined so as to see all records in the table or to see only records which meet specific logical criteria thus speeding up iterating through a table by a "filter" expression.

Getting Started

Installing Python CodeBase Tools

Users of Python for Windows, either 32-bit or 64-bit, can install this module directly from the Python Package Index (PyPI.org) using pip. The following command does the work if invoked in the command window from your <pythonroot>/Scripts directory:

```
pip install Code-Base-Tools
```

Once installed, the module can be instantiated thus:

```
From CodeBaseTools import cbTools
cbtObject = cbTools()
```

Opening, Closing, and Selecting a Table

The use() function works to connect to a DBF table and provides a mechanism for referring to that table in subsequent code. In the example below, the PRODUCTS.DBF table is opened by the use() function and assigned a user-specified alias name of "MYPRODUCTS". This will be used for future references to that table. The PRODCATG.DBF table is then opened without a custom alias name. In the absence of a user assigned alias name, the system assigns it an alias name equal to the base table name, in this case "PRODCATG".

The function alias() helps to keep track of which table is the currently selected table. This selection can be changed by the select() function call referencing the valid alias name of a currently open table. In the example in Figure 1 below, you are introduced to the reccount() function, which tells you the total number of physical records (including those marked for deletion) and the recno() function which indicate which physical record number is the current record in the currently selected table.

```
>>> from CodeBaseTools import cbTools
>>> oDB = cbTools()
>>> bTestOpen = oDB.use(r"E:\GIT_Repositories\Python-CodeBase-Tools\TestFiles\PRODUCTS.DBF", alias="MYPRODUCTS")
>>> bTestOpen
True
>>> oDB.alias()
'MYPRODUCTS'
>>> bTestOpen = oDB.use(r"E:\GIT_Repositories\Python-CodeBase-Tools\TestFiles\PRODCATG.DBF")
>>> bTestOpen
True
>>> oDB.alias()
'PRODCATG'
>>> oDB.select("MYPRODUCTS")
True
>>> oDB.reccount()
5
>>> oDB.select("PRODCATG")
True
>>> oDB.reccount()
3
>>> oDB.recno()
1
>>> oDB.eof()
False
```

Figure 1

The use() function illustrated in Figure 1 above has other parameters for greater control over file sharing which will be discussed later in this User Guide. Examples of the use() function are continued in the next section.

To close a single table opened by the use() function, use the table's alias name with the closetable() function. This function flushes all data changes from the record buffers to the disk, closes the table and its indexes, and unlocks any record or table locks related to the table as part of the process. To close ALL open tables, you call the closedatabases() with no parameters. All open tables are closed, buffered values are pushed to the disk, locks are unlocked, and select() functions referring to any of the closed tables will fail. In this case, the CodeBase Tool engine continues running and you can open other tables with the use() function normally.

Creating a Table and Adding Records

Let's start by opening a table, about which we don't know very much, but think may be empty. We want to add a record to that table, although we aren't sure exactly what the fields are in the table.

```
>>> from CodeBaseTools import cbTools
>>> oDB = cbTools()
>>> bTestOpen = oDB.use("products.dbf", alias="NEWPRODS")
>>> bTestOpen
True
>>> oDB.reccount()
0
```

Figure 2

The starting point in Figure 2, above is importing the cbTools object class from the CodeBaseTools module. Then an instance of the class is created, named oDB. The table is opened with the use() function. This requires a full path (in the example the code is being run in the directory where products.dbf exists) plus an optional "alias", a text string which identifies this table while it is open, allowing us to open other tables, and return to this one later. If the

bTestOpen value had been False, error message information would have been available. If no alias value had been supplied, a value of "PRODUCTS" would have been assigned automatically.

The final action in the example is to check to see how many records are in the table. The reccount() function returns this information as shown below in Figure 3. One way to determine what fields are in the table is to use the scatterblank() function, shown in the first line of code. This yields a dictionary of upper case field name keys, each of which has a value equal to the "empty" state of the field. So, for example, in the listing of the dictionary, the PROD_COUNT field (a numeric value) has a value of 0. The full content of the xRawRec dictionary is shown next:

```
>>> xRawRec = oDB.scatterblank()
>>> for cField, xValue in xRawRec.items():
...     print(cField, xValue)
PROD_KEY 0
PROD_CODE
PROD_NAME
PROD_CATG
PROD_COUNT 0.0
>>> xRawRec
{'PROD_KEY': 0, 'PROD_CODE': '', 'PROD_NAME': '', 'PROD_CATG': '', 'PROD_COUNT': 0.0}
>>> nNewKey = oDB.getNewKey(os.getcwd(), "PRODUCTS")
>>> nNewKey
11000
>>> xRawRec["PROD_KEY"] = nNewKey
>>> xRawRec["PROD_CODE"] = "ABDCode1"
>>> xRawRec["PROD_NAME"] = "Test Hardware Product"
>>> xRawRec["PROD_CATG"] = "HARDW"
>>> xRawRec["PROD_COUNT"] = 100
>>> oDB.insertdict(xRawRec)
True
>>> oDB.reccount()
1
```

Figure 3

The remainder of Figure 3 performs the record add function starting with the xRawRec dictionary of empty values.

There are several ways to add a new record, but the simplest and fastest is to use the insertdict() function, using a dictionary with the keys named for field names, and the item values being their corresponding desired field values. Since we didn't know the table structure, we called scatterblank() to create a dictionary with blank values for each field. (In just a moment, we'll see another way to interrogate the table to determine its structure.)

We notice that there is an integer field named PROD_KEY, which likely needs to be assigned a unique key. We have a table nextkey.dbf in the current directory, which is pre-configured to store the most recent unique value for a set of tables. We call the getNewKey() method to obtain that next key, and test it to find that the assigned value is 11000, the initial value assigned by the function for a newly added table reference. If the nextkey.dbf table is not found in the specified target directory, one will be created by getNewKey().

With that key information and knowing what we want to save in the table, we can populate the empty dictionary xRawRec with values for each field. Once it is populated, a simple call to insertdict() adds the record with all the values we have specified in xRawRec. If we had named a field which don't exist in the table, it would be ignored, unless there were NO matching fields, in which case an error would be raised.

Finally, we check to confirm that the record count is now 1.

As promised we now show how to see the structure of the table. Below in Figure 4 we see more information about the table structure:

```
>>> xFields = oDB.afields()
>>> for cFld in xFields:
...     print(cFld)
...
...

{'cName': 'PROD_KEY', 'cType': 'I', 'nWidth': '4', 'nDecimals': '0', 'bNulls': 'False'}
{'cName': 'PROD_CODE', 'cType': 'C', 'nWidth': '20', 'nDecimals': '0', 'bNulls': 'False'}
{'cName': 'PROD_NAME', 'cType': 'C', 'nWidth': '50', 'nDecimals': '0', 'bNulls': 'False'}
{'cName': 'PROD_CATG', 'cType': 'C', 'nWidth': '3', 'nDecimals': '0', 'bNulls': 'False'}
{'cName': 'PROD_COUNT', 'cType': 'N', 'nWidth': '5', 'nDecimals': '0', 'bNulls': 'False'}
>>> oDB.goto("RECORD", 1)
True
>>> xNewRec = oDB.scatter()
>>> xNewRec
{'PROD_KEY': 11000, 'PROD_CODE': 'ABDCode1             ', 'PROD_NAME': 'Test Hardware Product
TG': 'HAR', 'PROD_COUNT': 100.0}
```

Figure 4

In this code in Figure 4 we call the afields() function which returns a list of VFPFIELD objects containing descriptive information on each field. These objects have a custom str() implementation that formats the contents for console display with the print() function as shown in the iteration through the list. Another approach for human-readable structure information is the dispstru() function which creates a formatted text string that can be embedded in documentation for humans to read.

Finally, to confirm what we have entered into this first record, we call the goto() function with the "RECORD" parameter and specify record number 1. To retrieve the content of this record, we call the scatter() function, which returns a record-type dictionary similar to the one we used to create the record with insertdict(). In the illustration, we display the dictionary onto the Idle console (truncating part of the text on the right). Note that the code oDB.goto("RECORD", 1) moved the current record pointer to the first record in the table, i.e. record 1. Then the scatter() function operated on that record to yield a dictionary of field values which is displayed on the Idle console screen.

Adding (and removing) Indexes to Support Virtual Sorting

In the discussion of Navigational Table Access we explained how the default ordering of a DBF table is the physical record order – established by the sequence in which records were appended. But the option of one or more indexes was mentioned as a powerful feature of the system to establish alternate "orders" for searching and listing. While it is physically possible to

sort the record order of a table so that some field or combination of fields is the basis of the physical record-by-record sequencing of the table, it is rarely necessary to do so. Instead, in Python CodeBase Tools you apply an index to the table with a pre-defined ordering such that you can traverse the table record-by-record by that ordering instead of the raw physical ordering.

The system allows multiple indexes to be defined for a table such that when changes are made to the table records, all indexes are automatically updated to reflect the changes. All indexes are stored in a single .CDX file with the same base name as the data table, and the individual indexes in that .CDX are referenced by a "Tag Name" which is unique. The maximum number of tags allowed for a table is essentially unlimited – although each added index tag impacts update and add performance. The code below in Figure 5 illustrates features of the indexing capability. We start out by opening the PRODUCTS.DBF table with an alias name of NEWPRODS. In the use() function, the exclusive parameter is set to True. This causes the table to be opened in "exclusive mode", meaning that it may not be opened by any other process simultaneously. If another process had already opened the table, an error would be generated, since the exclusive mode could not be applied. The exclusive mode helps maintain data integrity while the state of the table is in transition.

```
>>> import os
>>> os.chdir(r"E:\GIT_Repositories\Python-CodeBase-Tools\TestFiles")
>>> from CodeBaseTools import cbTools
>>> oDB = cbTools()
>>> oDB.use("PRODUCTS", alias="NEWPRODS", exclusive=True)
True
>>> oDB.deletetagsall()
>>> oDB.dispstru()
5
E:\GIT_Repositories\Python-CodeBase-Tools\TestFiles\PRODUCTS.dbf
3

Structure for table:    E:\GIT_Repositories\Python-CodeBase-Tools\TestFiles\PRODUCTS.dbf
Number of data records: 3
Date of last update:    2026-06-12 20:34:27
Field  Field Name     Type     Width    Dec  Nulls

    1  PROD_KEY          I         4      0   No
    2  PROD_CODE         C        20      0   No
    3  PROD_NAME         C        50      0   No
    4  PROD_CATG         C         3      0   No
    5  PROD_COUNT        N         5      0   No
** Total **                      84
True
>>> oDB.indexon("PROD_KEY", "PROD_KEY", unique=15)
True
>>> oDB.indexon("PROD_CODE", "PROD_CODE")
True
>>> oDB.indexon("PROD_NAME", "UPPER(PROD_NAME)")
True
>>> oDB.indexon("PROD_COUNT", "STR(1000000-PROD_COUNT) + PROD_CODE")
True
>>> oDB.closetable("NEWPRODS")
True
>>> oDB.use("PRODUCTS", alias="NEWPRODS", exclusive=True)
True
>>> xIndexTags = oDB.ataginfo()
>>> len(xIndexTags)
4
>>> for iIndex in xIndexTags:
...     print(iIndex)
...
...

{'cTagName': 'PROD_CODE', 'cTagExpr': 'PROD_CODE', 'cTagFilt': '', 'nDirection': '0', 'nUnique': '0'}
{'cTagName': 'PROD_COUNT', 'cTagExpr': 'STR(1000000-PROD_COUNT) + PROD_CODE', 'cTagFilt': '', 'nDirection': '0', 'nUnique': '0'}
{'cTagName': 'PROD_KEY', 'cTagExpr': 'PROD_KEY', 'cTagFilt': '', 'nDirection': '0', 'nUnique': '15'}
{'cTagName': 'PROD_NAME', 'cTagExpr': 'UPPER(PROD_NAME)', 'cTagFilt': '', 'nDirection': '0', 'nUnique': '0'}
```

Figure 5

Prior to actually working on adding indexes, we make sure that all old index tags are removed from the .CDX file by calling the deletetagsall() function. Then to provide a listing of the table

structure for reference purposes while we work, we call the dispstru() function, which in this form simply writes a human-readable display of the table structure to the console.

In Figure 5, using the PRODUCTS.DBF table from the prior illustration, we have in the meantime added 2 more records to make things more interesting. In the last half of the illustration, we define 4 index tags that will, by default, be contained in the PRODUCTS.CDX file and be updated automatically as the contents of the table are changed. Note that once these tags are defined, the PRODUCTS.CDX index file will automatically be opened whenever the table is opened with the use() function and will be immediately available without any further action on your part. The four index definitions each illustrate a feature of the indexing capability:

- The PROD_KEY index is based on the integer PROD_KEY field, which we want to make unique and use for direct identification of a product record (primary key). The "unique" parameter is set to 15, which indicates that duplicate values of this field will NOT be allowed into the table. An attempt to save a record with a PROD_KEY the same as some other record in the table will generate an error condition and return a value of False. This is the reason for using the getNewKey() function in the previous examples. (CodeBase allows the definition of an auto-incrementing Integer field type, but that type is not compatible with other xBase applications, and is not implemented in Python CodeBase Tools.)

- The PROD_CODE field is an arbitrarily assigned string value that we likely will need to search for. You might decide to set it up with a "unique" property 15 also to enforce uniqueness, depending on your system data design. You can define multiple indexes as

  type 15. In the absence of an explicit setting, the default value of the unique property is 0, which allows any number of instances of the same value in the field.

- The PROD_NAME field contains a text string describing the product. At minimum, you may want to find PROD_NAME values with a case-insensitive search. To make that possible, the index value is "UPPER(PROD_NAME)", which indicates that all alphabetic characters in the field will be converted to uppercase when stored into the index. Note that an index may be based on values of one or more fields in the table with xBase functions transforming them for more complex ordering. Index expressions may be up to 240 characters in length, with an arbitrary number of fields and functions in the definition. (See also the Appendix: Expressions Allowed in Indexes and Searches.)

- The PROD_COUNT field is a numeric field indicating some kind of inventory quantity. The application requires that we list products in descending order by quantity with a "tie breaker" consisting of the PROD_CODE value. To combine a numeric value with the string PROD_CODE we use the STR() function (a standard xBase/FoxPro function) which translates the numeric argument to a 10-character long string. To that we append the

PROD_CODE value itself. The numeric argument for the STR() function is a large constant minus the inventory quantity so that when the index is traversed in natural order from low to high, the actual values of PROD_COUNT will range from high to low (reverse order).

Finally, we obtain a list of applicable index tags on the PRODUCTS table using the ataginfo() function. Testing the length of the list shows the correct value of 4. We can then list the indexes with an iterator, in which each item is a dictionary with index tag element names and associated values.

To illustrate how the index can work to iterate through the table in a defined order (other than the physical order of the records) see Figure 6 below:

```
>>> oDB.setorderto("PROD_NAME")
True
>>> oDB.goto("TOP")
True
>>> for xRec in oDB.scan():
...     print(xRec["PROD_KEY"], xRec["PROD_NAME"])
...
...
11001 Test Garden Product
11000 Test Hardware Product
11002 Test Home Appliance - Refrigerator
```

Figure 6

In the above example in Figure 6, we set the current "order" to the tag PROD_NAME and navigated to the top of the table based on that ordering (recalling the case-insensitive feature of the index.) We can then inspect all records in the table in the specified order using the scan() function which acts as an iterator across the table yielding a record dictionary for each record in turn – in this case xRec each time through the loop. We print out the PROD_KEY and PROD_NAME values for each record in the list of record dictionaries. Note that the list is in alphabetic order by the PROD_NAME, ignoring the case of those values.

The scan() function has a number of optional features, including the choice of limiting the fields returned, limiting the records returned to those that match a logical expression, and even returning only a record number for speed purposes. During the iteration, the "current record" in the table is the one contained in the field dictionary, so you have the option of inspecting individual field values, changing field values, and using field values to search for records in other tables (which we will illustrate more completely later.) You can learn more about scan() from the HTML help provided in the GitHub repository or from the embedded documentation in the CodeBaseTools.py module.

Deleting Records and Clearing Tables

The xBase tools for managing DBF data tables have a specialized approach to the deletion of records. Every record in a DBF table has a "deletion flag". The delete() function applied to the

current record of an open currently selected table will turn the deletion flag to True. However, deletion flag values are NOT necessarily automatically respected when searching for or scanning records. There is a global "deleted status" state which can be "True" where deletion flag values are respected (and deleted records do not appear in scanning or search results) or "False", in which case the deletion flag value is ignored and records are visible regardless of their deleted setting.

The global state of the deleted status is set by the setdeleted() function. Passing a True value sets the deleted status to true for all subsequent actions in CodeBaseTools and all tables both those already opened and those opened subsequently. Passing a False value turns off deleted status universally as well. Passing a None as the parameter returns the current state of the deleted status flag for the system. Regardless of the setting of the deleted status in the system, the deleted() function will return the deletion flag value for the current record of the currently selected table, so you can test it and possibly adjust your code logic accordingly.

After a while, active tables may accumulate large numbers of deleted records, which may result in some performance impact. The system provides a pack() function which will remove all records from the table which have the deletion flag set to true. In the process, it will reindex all the index tags. You should be careful to run pack() only on tables you have opened exclusively since any access by another process while a pack is under way can result in faulty data results and or corruption. *There is another caution related to pack() operations: when deleted records are removed they are removed completely, and the record numbers of the surviving records may be changed as a result. While searching by record number is extremely fast, it may be unreliable if the numbers being searched have been stored for some time and a pack() may have been executed in the meantime!* This is a strong rationale for having an integer primary key value in the table which will never be altered by pack() operations and which can be searched for very quickly.

There are two functions that can undo a deletion. The recall() function will remove deletion flag status from the current record of the currently selected table. But note, if the deleted status is set to True, the system will avoid selecting or finding a deleted record. In that case, you will need to temporarily set the deleted status to False, so the record can be seen by the application, and then call the recall() function before setting the deleted status back to its previous state.

The opposite of the pack() function, which removes all deleted records, is the recallall() function which sets the deletion flag to False on all records in the table, regardless of the setting of the deleted status. As with pack() it is wise to apply this function only to tables opened with the exclusive setting of True.

In the most extreme case of deleting records, you may want to remove ALL records in a table. That is accomplished by the zap() function. Be careful! The zap() function removes all records from the table, and once they are gone, they are not recoverable. The table will be empty and

no backup will be created unless you handle that in your code. Of course, with something so drastic, the zap() function will not be allowed to be executed unless the table has been opened in exclusive mode.

Selecting and Finding Records

Selecting one or more records based on some logical criteria and performing some action on each is one of the most common tasks you will likely perform with CodeBase Tools. There are several ways to search for specific records, depending on whether you expect to search using an index, and whether you expect a single record to be found or possibly multiple records matching your criteria.

Finding a Single Record

In this case you either know there is one and only one matching record, or you don't care if there is more than one, you just want the first one found. In our example table PRODUCTS.DBF, we've added a few more records to make the searching more interesting.

First, you may want to satisfy yourself that there's only one record that matches your search criteria. You can use the countfor() function to accomplish this after opening your table as shown below in Figure 7Figure 7.

```
>>> oDB.use("PRODUCTS")
    True
>>> nCount = oDB.countfor("PROD_CODE='HOMEAPP1'")
>>> print(nCount)
    1
```

Figure 7

The logical expression found just one matching record in the above case. Note the single equal sign ('=') in the expression. The search found one record where the PROD_CODE field began with the letters 'HOMEAPP1'. But note that the defined field width is 20 characters. In the DBF format, all plain character fields are automatically padded with blanks to their specified width. If you had used an expression of "PROD_CODE=='HOMEAPP1'", the result would have been 0 count, since the == sign demands an exact match between your target string and the value of the field. If you had a reason to test the full 20 characters of width, you could have used the expression "PROD_CODE=='HOMEAPP1'+SPACE(12)", which would get an exact match.

Using the countfor() function is usually superfluous, and instead we would go directly to the search for the record using the locate() function as demonstrated in Figure 8 below:

```
>>> bTestLookup = oDB.locate("PROD_CODE='HOMEAPP1'")
>>> print(bTestLookup)
True
>>> xRec = oDB.scatter()
>>> xRec
>>> for cFld, xVal in xRec.items():
...     print(cFld, xVal)
...
...
PROD_KEY 11002
PROD_CODE HOMEAPP1
PROD_NAME Test Home Appliance - Refrigerator
PROD_CATG HOM
PROD_COUNT 3.0
```

Figure 8

You invoke the locate() function with the same logical expression you used for the countfor(). It found a record and returned a value of True. To get the currently selected record content, we used the scatter() function, which returned a record dictionary, which we then listed to the screen using the items() iterator. It displays the PROD_CODE value we were looking for in Figure 8Figure 8. In the real world, you'd add a test for the return value from the locate() function and only perform the resulting scatter() if a True value was returned.

The locate() function does not depend on any indexes (although it will try to use one if possible) and can be called repeatedly (using the locatecontinue() function) to see if there are additional records that match the original criteria as shown in Figure 9 below:

```
>>> bTestLookup = oDB.locatecontinue()
>>> bTestLookup
False
>>> print(oDB.cErrorMessage)

>>> oDB.locateclear()
True
...
```

Figure 9

Once you are finished calling locatecontinue(), you need to call locateclear() which resets the current default selection criteria for your next call to locate(). (Some other functions, like countfor(), require that any prior locate() calls have been cleared with locateclear() for them to work correctly.)

```
>>> bTestLookup = oDB.locatecontinue()
>>> bTestLookup
False
>>> print(oDB.cErrorMessage)

>>> oDB.locateclear()
True
...
```

Figure 10

In our example in Figure 10Figure 10, the locatecontinue() returned False as there were no more matching records. We tested for an error condition just in case – nothing was returned, indicating that there legitimately were no further matching records and the False was not triggered by a logical error in the execution. Finally, we issued the locateclear() function.

While the above logic using locate() works under an array of conditions which may or may not suggest use of an index, for large tables the exhaustive search for matching records may take several seconds. If there is an index that can be used for a direct search, you can exploit the high performance of index lookups. In this case, we know from previously using ataginfo() that there is an index tag referencing PROD_CODE. We can use that to find the record we want very quickly as shown in Figure 11Figure 11 below:

```
>>> oDB.goto("TOP")
    True
>>> oDB.setorderto("")
    True
>>> bTestLookup = oDB.seek("HOMEAPP1", "PRODUCTS", "PROD_CODE")
>>> bTestLookup
    True
...
```

Figure 11

To demonstrate how this works we first went to the top of the table and cleared any index ordering. This was for illustration only, and not required for this code to work. Using the seek() function, we specified a text string to match in the PROD_CODE field, specified the alias name of the table (in this case 'PRODUCTS'), and specified the tag name of the index to use. The result was a True, as there was a record matching that index lookup. When the function returned, the record pointer was positioned on the appropriate target record. There was no need to repeat the display of the record contents for this illustration. Next, we do a search on a numeric primary key field in Figure 12Figure 12.

```
>>> oDB.goto("TOP")
    True
>>> bTestKey = oDB.seek("11003", "PRODUCTS", "PROD_KEY")
>>> bTestKey
    True
>>> xRec = oDB.scatter()
>>> for cFld, xVal in xRec.items():
...     print(cFld, xVal)
...
...

    PROD_KEY 11003
    PROD_CODE DFG_THING
    PROD_NAME Another Garden Product
    PROD_CATG GAR
    PROD_COUNT 30.0
```

Figure 12

In this example above in Figure 12Figure 12 we searched for the PROD_KEY value. This field is an integer. But note that the search expression was the integer converted to a Python string (we could have used str() on a numeric variable). This is a feature of CodeBase Tools. If an integer value is passed in this parameter, a Python ValueError will be generated. In this example, the function returned True, and displaying the content of the record shows that the PROD_KEY value is the desired 11003.

Finding Multiple Records Matching Your Criteria

You've already seen how the locate() function's search can be repeated to retrieve multiple records by calling the locatecontinue() function. But, as indicated, it can have performance issues on large tables. There are other search techniques that provide better performance when multiple records are to be retrieved by some criteria.

The copytoarray() function offers some convenience features and a small performance improvement over the repeated locate() search, in that the function returns a list of matching record dictionaries through which you can iterate as you need to without re-searching for individual records in the table. Figure 13 below shows how this is done.

```
>>> xRecs = oDB.copytoarray("PRODUCTS", fieldtomatch="PROD_CATG", matchvalue="GAR", matchtype="=")
>>> for x in xRecs:
...     for cFld, xVal in x.items():
...         print(cFld, xVal)
...
...

PROD_KEY 11001
PROD_CODE DFG_CODE
PROD_NAME Test Garden Product
PROD_CATG GAR
PROD_COUNT 50.0
RECORDNUMBER 2
PROD_KEY 11004
PROD_CODE DFG_PLANTER
PROD_NAME Garden Planter
PROD_CATG GAR
PROD_COUNT 5.0
RECORDNUMBER 5
PROD_KEY 11003
PROD_CODE DFG_THING
PROD_NAME Another Garden Product
PROD_CATG GAR
PROD_COUNT 30.0
RECORDNUMBER 4
```

Figure 13

The example above in Figure 13 shows copytoarray() searching for records with a PROD_CATG value of "GAR". It returned a list of record dictionaries xRecs which we listed in nested iterators. There are optional parameters for copytoarray() which can limit the total number of records returned, set a list of field names to return, and specify if the end-of-field spaces should be trimmed off character fields. Since all of these constraints are applied at the C language API level, they perform more quickly than Python-only code using the locate() subsystem. Note that there is an additional feature of the copytoarray() function in that it adds a "field" to the record dictionary named "RECORDNUMBER" with the physical RECNO() value of the record from the table. This allows you to find those records again very quickly in your code to perform further operations.

Finally, even though the seek() function finds just one record, there is a coding pattern that uses seek() and skip() to retrieve matching records much more quickly, if an appropriate index exists, than would be possible with a locate() series or copytoarray(). An example of how this works is presented below in Figure 14 using the PRODUCTS.DBF table once again.

Referring to Figure 14Figure 14, we know that there is an index tag on the expression PROD_CATG, so we can use that to retrieve all the records with a value of “GAR” in the PROD_CATG field. After opening the PRODUCTS table, we start with setting the current order to “PROD_CATG”. That tells the system to use that index when asked to traverse the table in “order”. Then the goto(“TOP”) function uses that index to find the smallest value of PROD_CATG and move the record pointer to that “top” record.

```
>>> bTestOpen = oDB.use(r"E:\GIT_Repositories\Python-CodeBase-Tools\TestFiles\PRODUCTS.DBF")
>>> oDB.setorderto("PROD_CATG")
    True
>>> oDB.goto("TOP")
    True
>>> reclist=list()
>>> bFirstTest = oDB.seek("GAR", "PRODUCTS", "PROD_CATG")
>>> bFirstTest
    True
>>> if bFirstTest:
...     reclist.append(oDB.scatter())
...     while True:
...         oDB.skip(1)
...         xRec = oDB.scatter()
...         if xRec["PROD_CATG"] != "GAR":
...             break
...         if oDB.eof():
...             break
...         reclist.append(xRec)
...         continue
...
...
    True
    True
    True
>>> for xFound in reclist:
...     for cFld, xVal in xFound.items():
...         print(cFld, xVal)
...
...
    PROD_KEY 11001
    PROD_CODE DFG_CODE
    PROD_NAME Test Garden Product
    PROD_CATG GAR
    PROD_COUNT 50.0
    PROD_KEY 11003
    PROD_CODE DFG_THING
    PROD_NAME Another Garden Product
    PROD_CATG GAR
    PROD_COUNT 30.0
    PROD_KEY 11004
    PROD_CODE DFG_PLANTER
    PROD_NAME Garden Planter
    PROD_CATG GAR
    PROD_COUNT 5.0
```

Figure 14

Continuing with Figure 14, we then create an empty list which will contain our record dictionaries. Then to start the search we call a simple seek() function to find a matching record by the PROD_CATG index. We test the result of that seek(). If it is True, we have found at least one record with “GAR” and we can continue to look for more, which should be consecutive records when traversing by this index. If it is False, we give up.

There is a “GAR” found, and since we are working with the table as if it were physically ordered by the PROD_CATG field values, the seek() finds the top-most record in the table having that desired PROD_CATG value of “GAR”. We saw that the seek() returned True, so we appended the current record into the reclist list(). To load the rest of the matching records, we use a while construct in which we call skip(1) repeatedly. It scans down through the table in PROD_CATG order. If an end of file is encountered, we break. If a value of PROD_CATG is found in the current record which is NOT “GAR”, we know we are done (the table is in PROD_CATG order, remember) and we break. If those conditions are not met, we have another matching record which we retrieve and store into reclist.

Finally, after the while loop is completed in Figure 14, we list the contents of the reclist list() to show that indeed we have records which all have a PROD_CATG value of “GAR”. For large tables (say 100,000 records or more) this construct will execute many times faster than the locate() functions so long as a suitable index is available. You may find that with the speed of index generation, it may even pay to apply a temporary index, extract the target records using this construct and then remove the index to achieve greater performance on large tables than with locate(). See below under Special Topics for more on temporary indexes.

Retrieving/Changing/Updating records

Single User Scenario

With many applications there will never be more than one process accessing the tables at one time. In these applications special logic to protect multiple users is not required, and the programmer is free to add, retrieve, and update records without thinking about other users who may also be accessing the same tables or even the same records in those tables.

In the Selecting and Finding Records section above you have seen the use of the scatter() function to retrieve the currently selected table’s current record into a record dictionary where the keys are upper case field names, and the values are the Python variable types associated with the data field type. The scatter will also allow access to the current record of other tables currently open by simply passing the alias name as a parameter. For example: xRec = oDB.scatter(alias=”PRODUCTS”), which by default converts every field in the table to its corresponding type of Python variable.

However, for performance reasons, you may wish to limit your data retrieval to just one field or a selection of fields (remember that CPU cycles are consumed when DBF field values are converted to their Python variable types). The scatter() function has an optional parm “fieldList” which can be passed as a comma-delimited list of field names (in upper case) to include in the output. But, the fastest performance is achieved by calling functions that already know the type of field value they are retrieving and avoid all the overhead of type checking. The functions that do this are listed below:

- curval() – Pass any field name (if not in the current table, prefix with the table alias name, e.g. “PRODUCTS.PROD_CATG”). This function does NOT convert the type automatically, unless its convertType parameter is True. If not converted, the field content is returned in a two value tuple with the value as a string, and a code indicating the type. This form of the curval…() functions would normally only be used if you aren’t certain what the type of the field value is.
- curvaldate() – Operates only on date or datetime fields. Returns a Python datetime.date object or None if the field is empty or the field name is not found or the table is not accessible. (DBF format supports empty date and datetime fields.) If None is returned, you may wish to check the cErrorMessage property to confirm that nothing went wrong and the None is a legitimate value from the table.
- curvaldatetime() – Operates only on date or datetime fields. Returns a Python datetime.datetime object or None if the field is empty or an error has occurred. If the field is of type date, the hour, minutes, and seconds properties of the result will all be 0. See above for verifying the None value.
- curvaldecimal() – Operates only on number, float, or currency type DBF fields. The values are converted to Python Decimal objects with fixed numbers of decimal places.
- curvalfloat() – Operates only on number, float, double, or currency type DBF fields. Returns a Python float value.
- curvallogical() – Operates only on logical type DBF fields. Returns True or False. None is returned on errors like table not open or no such field name.
- curvallong() – Operates on integer or number type DBF fields, but note that if used to retrieve a number field value, the value will be truncated to an integer value.
- curvalstr() -- Works OK with char, memo, number, currency, and date type fields. Does not produce useful results from integer, double, datetime, or float type DBF fields.
- curvalstrEX() – Applies to character fields but offers an optional parameter to strip off trailing blanks in character fields. Not recommended for char (binary) or memo (binary) fields.

To store data into a table, you have multiple options. You’ve already seen the use of insertdict() in the Getting Started section where a record dictionary is populated by your program and then stored into a new record in the currently selected table or the table referenced by the “alias” parameter. But you may also need to update a record either completely, or update individual fields.

The gatherdict() function takes an optional “cAlias” parameter (if left empty or omitted, the action is performed on the current record of the currently selected table) followed by the

object name of a record dictionary. Every field in the table with a matching key found in the dictionary will have its field value updated to reflect the value in the dictionary. Fields NOT found in the dictionary will be ignored. The returned value is True on success, otherwise False.

If you just want to update an individual field with a new value, the replace() function takes a field name (upper case) to replace and a parameter with the new value for the field. It ONLY works on the current record of the currently selected table. All field types can be handled by this function, although you should take care to pass compatible values. In other words, passing a character value to an integer field will typically generate a Python error message.

There is a special, high-performance function replacedatetimeN() which replaces the specified datetime field in the table using numeric parameter values for year, month, day, hour, minute, and second, thus avoiding some of the data conversion overhead.

Note that changes to currently open tables are typically buffered by CodeBase Tools using as much as 100 MB of buffer capacity. This provides for great performance. When you close the tables with the closetable() function, these buffers are written out to the disk. Since this is the Single User Scenario, that should not be a problem for you. (But if your application is to be run on a computer or server subject to unplanned shutdowns or re-starts, you should make liberal use of the flush() and flushall() functions to ensure your changes are written to disk immediately. And you should close tables as soon as your work is done with them. All this to avoid possible data loss and corruption!)

Multi-User Scenario

Things get much more complicated as soon as multiple processes (or users) may be expected to access your data tables at the same time as your process is reading or updating values in those tables. Simple SQL database systems often don't provide very good tools for dealing with this situation, possibly leading the corrupted data and/or incomplete data inquiries. (SQLite is a great example, as it doesn't support record-level locking at all and attempts to lock the entire table on any update!) CodeBase Tools provides a suite of record and table locking functions plus buffer management tools to make sure reading and writing of the file data is handled smoothly for all users.

Protecting Reading Records

In the simplest case of reading some field values for subsequent processing, a simple data retrieval using scatter() or a curval() function may be fine. However, for instances where your application is reading records while other processes may be writing to them, it is possible that your read buffers will not be up to date with the latest changes made by others. To protect against this condition, immediately before retrieving the record value(s), you should call refreshrecord() assuming that you have found the desired record already in the currently selected table. But… in cases where there can be many changes, including record additions, which may not appear in your local table buffers, you can call refreshbuffers() which clears out

all data buffered for the selected table and forces a re-read of the current data from the disk during the retrieval process. This helps ensure you are working with the latest data from the table, but it may be at a price of the performance hit from re-reading and storing data into the local buffers.

Locking for Reading and Writing to Tables

CodeBase Tools automatically locks the current record as it's being loaded into the current record buffer. The lock is then automatically removed once the record is read. You can override this behavior by calling rlock() immediately before calling scatter() or one of the curval() functions. You should then call unlock() explicitly.

When you make changes to a current record using gatherdict() or replace(), no locking is performed immediately. Your changes are saved in the local record buffer. By default, the system attempts to apply a momentary record lock to the current record when you call some function that moves the record pointer off that current record. Examples would be any one of the goto() function forms, skip(), etc. Your record change code may fail with the current record pointer unchanged, if there is already a lock on the record placed by another user or if you have changed a field with a unique (The setting is 15 in the indexon() function, see the PROD_KEY index example in Figure 5) index on it and the new record has a unique field with a value that is already in the table in another record.

In applications with lots of users/processes accessing tables and records, you can take charge of locking in your code to protect against locking conflicts. To lock a record you want to update, you can call the rlock() function which operates on the current record of the currently selected table. You can specify the number of re-tries and the interval between them, anticipating that if another user has the record locked already, that user will unlock immediately after any updates are completed, at which point rlock() will return True and you can proceed. If necessary, you can call rlock() repeatedly until it succeeds before ultimately giving up and handling the update failure in your error trapping code.

Once rlock() is successful, you can update the record, issue a flush() function call, optionally move to a different record, and then call unlock().

In more extreme situations where you may need to update multiple records safely without other processes interfering, you have the option of issuing a flock() function, which applies the lock to the entire table. Like rlock(), flock() allows retries and interval settings. Use this VERY SPARINGLY, and unlock as soon as processing is done to avoid performance hits to other processes.

A special situation occurs when you want to append a record to the table. In the single user situation, there is nothing to worry about. You just call insertdict() with your data. So long as your unique index settings are respected, the function will execute in millisecond time and return True. But with multiple users, the table header must be protected during the few

milliseconds when the table size is expanded, the record count is incremented, and the table is prepared for updating the new record with your data. To accomplish this, you should use the appendlock() function (with its retries and interval properties) immediately before calling insertdict(), and unlock() immediately after insertdict() returns successfully. It is very important to keep the appendlock() in place for as short a time as absolutely possible. If your code requires a bunch of processing to assemble all values for your newly added record, you should complete all that work before the appendlock()/insertdict()/unlock() series to minimize impacts on other users.

Exporting to and importing from CSV (and Other Formats)

Standard CSV Export and Import

One of the most widely used universal data file formats is the "Comma Separated Value" type, where the file name extension is either .TXT or possibly .CSV. If the CSV data is properly behaved, meaning that each line of text has the same number and ordering of fields, it is possible to use CodeBase Tools to import the data quickly into a DBF table. Similarly, any DBF table can be written out to a CSV format by CodeBase Tools. Due to its ubiquity, the Excel spreadsheet version of CSV is the default one read and written by CodeBase Tools. But other dialects of CSV can be handled as well. The example below in Figure 15 shows how this works:

```
>>> from CodeBaseTools import cbTools
>>> oDB = cbTools()
>>> oDB.use(r"E:\GIT_Repositories\Python-CodeBase-Tools\TestFiles\products.dbf")
True
>>> oDB.alias()
'PRODUCTS'
>>> bTestOutput = oDB.copyto(cAlias="PRODUCTS", cOutput=r"E:\GIT_Repositories\Python-CodeBase-Tools\TestFiles\PRODUCTS.CSV",
cType="CSV")
>>> bTestOutput
5
>>> bTestOutput = oDB.copyto(cAlias="PRODUCTS", cOutput=r"E:\GIT_Repositories\Python-CodeBase-Tools\TestFiles\PRODUCTS2.CSV"
, cType="CSV", bHeader=True)
>>> bTestOutput
5
```

Figure 15

In the example code above, Figure 15, we open the PRODUCTS.DBF table and output it in two different formats of CSV. The first call to copyto() specifying cType = "CSV" leaves the optional parameter bHeader at its default False value. The result is the output shown below in Figure 16:

```
11000,"ABDCode1          ","Test Hardware Product                    ","HAR",100
11001,"DFG_CODE          ","Test Garden Product                      ","GAR",50
11002,"HOMEAPP1          ","Test Home Appliance - Refrigerator       ","HOM",3
11003,"DFG_THING         ","Another Garden Product                   ","GAR",30
11004,"DFG_PLANTER       ","Garden Planter                           ","GAR",5
```

Figure 16

Changing the copyto() call to set bHeader True produces the slightly different format in the result below in Figure 17:

```
PROD_KEY,PROD_CODE,PROD_NAME,PROD_CATG,PROD_COUNT
11000,"ABDCode1            ","Test Hardware Product                          ","HAR",100
11001,"DFG_CODE            ","Test Garden Product                            ","GAR",50
11002,"HOMEAPP1            ","Test Home Appliance - Refrigerator             ","HOM",3
11003,"DFG_THING           ","Another Garden Product                         ","GAR",30
11004,"DFG_PLANTER         ","Garden Planter                                 ","GAR",5
```

Figure 17

Here in Figure 17 the first line of text is a comma separated list of the field names contained in the data.

Importing CSV data into .DBF tables makes use of the "Swiss Army Knife" of functions, appendfrom(). The full documentation for all the parameters for this function is found in the doc_string in the CodeBaseTools.py module. In brief, if the CSV file has field names in the first row (as illustrated in Figure 17) or you pass a comma delimited list of fields in the parms, it is possible to let the importer map the data into the fields of the DBF table. If the DBF table has fields not given in the CSV file, they will be filled with "empty" values, if the CSV file specifies fields not in the DBF table, they will be ignored. If no fields match, an error message will be generated.

Customizing CSV Imports

Many systems claim to output CSV data, but differ somewhat from the widely used Excel dialect of CSV and require customized handling by the appendfrom() function to properly read the data from the text file. The function offers several parms to allow its interpretation of things like quoted text and end-of-line handling, but for greatest flexibility, you can define all of these behaviors (and more) by exploiting the Python csv.Dialect class and its many properties. With this as your base class, you can define a subclass with customized properties and can then assign an object reference from your custom Dialect class to the CodeBase Tools module's oCsv.Dialect property. For more details on customization, see the Python documentation on the csv module, part of the standard Python library.

Other Imports and Exports

The two complementary functions copyto() and appendfrom() support formats other than CSV as well. The most straightforward format is "DBF" through which you can copy any table to another table (copyto()), optionally limiting the fields and records output, and then retrieve the output data (appendfrom()) into yet another DBF table with a matching field structure.

The other two recognized formats are "TAB" and "SDF". The TAB format is much like CSV except the individual fields are separated by Tab characters (ASCII 9 values). This simplifies the situation where special characters (like quote signs) may be found in the middle of text character fields. The SDF format (the type name referring to the ancient "System Data Format" designation) defines records in text format with each record separated by a CR/LF combination (or just LF in some cases) such that each field is represented by a fixed length text string. If the

field text string lengths align with the widths of fields in a DBF table, the data can be appended into the DBF file with the appendfrom() function. Note that embedded CR/LF characters in text fields within the source DBF data table will be removed automatically when copyto() is executed to produce either TAB or SDF format outputs. For comprehensive documentation, see the doc_string values for the copyto() and appendfrom() functions in CodeBaseTools.py.

HINT for all imports: If the target DBF table has indexes with unique settings (as for primary key fields) an attempt to use appendfrom() may fail unless the source CSV data has pre-assigned unique keys which aren't duplicated in the DBF target table. If this isn't assured, you can create an empty table with no indexes into which to import your CSV data, and then use a Python script to populate the primary key values in anticipation of importing the result into the target table with appendfrom()

Searching, Navigating and Iterating through Related Tables

The underlying CodeBase C .DLL module allows the definition of "relations" between tables where a "remote key" in a table is defined as mapping to a "primary key" in another table. In this definition, one table is defined as the "parent" or "main" table, and an iteration through that table will result in related records in related tables being made current simultaneously. While convenient in theory, the implementation in CodeBase is complex, and this feature has **not** been enabled in CodeBase Tools. However, the powerful functions like seek() and copytoarray() are available to help you easily code the equivalent capability. See the section below Working with Multiple Related Tables.

Table Introspection and Structure Definitions

Unlike some SQL systems, CodeBase Tools provides functions to determine the field structure and index composition for any table programmatically. This allows dynamic, programmatic response to the characteristics of previously unknown tables. In this section we cover both the mechanisms for determining field and index features, but also cover how the system converts VFP and DBF fields to Python, both for Python 2.7 and Python 3.x and up.

Table and Field structures… afields(), afielddict(), and afieldtypes()

The basic tool for retrieving the structure of a table in CodeBase Tools is the afields() function which was illustrated in Figure 4. This function interrogates the currently selected table, or the table indicated by its lpcAlias parameter. It returns a list of VFPFIELD objects, one for each field in the table in their correct order. There are two other convenience functions: afielddict() with an lpcAlias parameter which returns a dictionary where each field is represented by its upper case name as a key, and the value is the definitional VFPFIELD object. For rapid retrieval of just simple field type information, the afieldtypes() returns a simple dictionary keyed by uppercase field names and the values being a single character string indicating the CodeBase Tools field type.

The VFPFIELD class properties/attributes are as given in the definition below:

class VFPFIELD(object):

This is a simple quasi-structure class that contains attributes for all properties required for creating a field in a CodeBaseTools .DBF table.

Attributes:

cName: The text string of the field name. Typically, upper case and a maximum of 10 characters in length. Must start with a letter or underscore and may contain only alphanumeric characters and underscores. (See the Appendix on Capacities and Limits for a Visual FoxPro compatibility exception.)

cType: The CodeBase letter code for the type of field. See below in this section.

nWidth: The width of the field. This is used for character fields (type 'C') and number fields (type 'N'). For character fields, this is the fixed width of the field in bytes. If you'll be storing Unicode data, which may have multi-byte values, be sure to allow enough room. For Number fields, this specifies the total number of digits to make room for. For example, if the largest value you expect to store in the field is 999.99, then you should set the width to 6 to provide room for digits on both sides of the decimal point AND for the decimal point itself.

nDecimals: Used for number fields (type 'N'), this specifies how many digits to the right of the decimal point to allow for. May be 0. Not required for integer type fields. Where not relevant, defaults to 0.

bNulls: By default, .DBF table fields may not take on a value of NULL ('None' in Python code). NULL in .DBF terminology is truly "unknown". However, if a data field must be able to store a value of NULL, then set this value to True. Note that in a DBF table, DATE and DATETIME fields may contain a DBF table value of [EMPTY], which is NOT the same as NULL. When CodeBase Tools retrieves a DATE or DATETIME value containing [EMPTY], it is converted to a Python None value even if that field is defined with a bNulls value of False.

Field Type Mapping to Python Variables

Many CodeBase Tools functions return table data automatically converted from DBF table field types to standard Python variable types. The following table in Figure 18 shows how DBF field types are converted to Python types. Note the differences between Python 2.x and 3.x, which recognize the major change in Python in string storage and handling between versions:

VFP Type Code	DBF Type Name	CodeBase Type Code	Python 2.x Type	Python 3.x Type
B	Double	B	float	float
C	Character	C	str	str (Unicode)
C	Character (binary)	Z	bytearray – Note 3	bytes

D	Date	D	datetime.date	datetime.date
F	Float	F (same as Numeric)	float	float
G	General	G – Note 2	bytes	bytes
I	Integer	I	Int or long	int
I	Integer (auto increment)	N/A Note 1	N/A	N/A
L	Logical	L	bool	bool
M Note 4	Memo	M	str	str -- Unicode
M Note 4	Memo (binary)	X	bytes	bytes
N	Number	N	float	float
Q	Varbinary	Not supported	N/A	N/A
T	Datetime	T	datetime.datetime	datetime.datetime
V	Varchar	Not supported	N/A	N/A
V	Varchar (binary)	Not supported	N/A	N/A
W	Blob	Not supported	N/A	N/A
Y	Currency	Y	Decimal	Decimal

Figure 18

Note 1 -- The last Codebase version added support for an auto incrementing integer field, but these fields are NOT recognized by Visual FoxPro, and VFP auto incrementing integer fields are not recognized by Codebase and can cause Codebase to crash. This module therefore does not support auto incrementing integer fields directly, but a utility function getNewKey() is provided as a source for consecutive integer values for each named table.

Note 2 -- CodeBase and this module support G (General) type fields for reading and writing, but unlike in Visual Foxpro, their contents cannot be filled with some kind of APPEND GENERAL sort of function accepting OLE or COM type objects (like an Excel spreadsheet). VFP tables which have had a G field populated by the APPEND GENERAL command from an OLE document can be copied to other tables by this module, however, as these fields are read as simple strings of generic binary data (possibly containing null bytes), exactly as X Memo(binary) fields, there is no interpretation done with them or ability to extract their underlying document type by OLE services.

Note 3 -- All "strings" in Python 3.x are Unicode. Sequences of bytes that are not intended to represent text of any kind are stored in "bytes" variables, not "strings". Accordingly, for Python 3.x the default Python type target for a standard char or memo field is a string, and that of char or memo binary fields is bytes. If you are interoperating with a Visual FoxPro application, you may want to store Unicode data in binary fields to ensure no CodePage conversions are attempted by FoxPro. As a result, options are provided to write to and read from binary fields in several different Unicode codings. See the section Special Considerations for Storing Python Strings on page 44.

Note 4 – The contents of Memo fields of both types are stored in separate .FPT files with the same base name as the DBF table itself. There is a link to the file and the content record blocks in the .FPT files in 8-byte fields in the DBF table. The contents of Memo fields can be indexed, with some limitations.

Field and Tag Name Conventions

Field Names – Seven-bit ASCII is the default coding (and Python 3.x strings will be interpreted as such by CodeBase Tools functions, but extended ASCII with Code Page 1252 is supported for displaying table structure. Fields having names with characters above ASCII 127 cannot be accessed for reading from and writing to tables.

Index Tag Names – Seven-bit ASCII is the default coding, but extended ASCII tag names work properly. Maximum size is 10 characters.

Alias Names – Must be standard seven-bit ASCII strings. If specified with cp1252 high order bit characters, the Alias Name will be coerced to 7-bit ASCII. When a table is opened without specifying an alias name in the use() function, an automated alias name will be assigned consisting of the base name of the DBF file, adjusted to correct for embedded spaces, punctuation, and numeric leading characters, which are not allowed in alias names.

Table Names – Character text type names supported by the operating system will work, but filename type objects are not accepted in the use() function.

Index tag definitions… ataginfo()

By default, CodeBase Tools stores index data in index files with a .CDX extension and a base name identical to that of the DBF table. A single .CDX file may contain a large number of distinct indexes, each identified by an Index Tag Name. Opening a .DBF table with an associated .CDX file will automatically open that index file and read the tag information contained in it. An open table with a .CDX file and index tags may have one index currently in force for iterating forward and back through the table. If no index is current, then iterating through the table is in record number order. Regardless of whether an index Tag is currently active, all indexes are automatically updated when values in the table are changed. The ataginfo() function returns a list of index VFPINDEXTAG objects for the currently selected data table and its .CDX file.

The VFPINDEXTAG object properties/attributes are as given below:

class VFPINDEXTAG(object):

This is a simple quasi-structure class that contains attributes for all properties required to create an index tag:

Attributes:

cTagName: The text string containing the uppercase name of the index tag. Must be unique for the table and be no more than 10 characters in length.

cTagExpr: The text string containing the expression being used to define the index order. This may be empty or an arbitrarily complex expression involving both field names, constants, and internal CodeBase (VFP) functions (See the Appendix). Maximum size is 240 bytes.

cTagFilt: A string containing a logical expression which will limit the records which will be considered by this index tag. If this is defined, then an iteration through the table when this index is active will see only records which meet the cTagFilt logical expression.

nDirection: 0 indicates ascending, 1 specifies descending.

nUnique: One of several possible values which define handling of cases where the value in the cTagExpr may or may not appear in multiple records. Allowable values are:

- 0 = Duplicate records allowed in index expressions
- 15 = Behaves like VFP "Candidate" indexes where attempting to save a record for a duplicate key generates an error. Behaves like a Primary Key index in VFP, which CodeBase per se does not support.
- 25 = Behaves like a VFP "Unique" index which ignores records with duplicate keys after the first one is encountered.
- 20 = Special CodeBase functionality, but behaves like 25 when opened by VFP.

Working with Multiple Related Tables

In the absence of a SQL evaluation module as found in Visual FoxPro, your applications will need to handle multi-table searches and scanning explicitly in your code. However, this means you have complete control over how tables are scanned, what associated tables are searched for relevant records, and which search methods you prefer – balancing speed against program simplicity. In the sections above in this document, you've seen several approaches to iterating through a table in search of specific records (or all records). Here we show some examples of combining those techniques to gather information from multiple related tables.

[Note – Advanced topic] The underlying CodeBase product provides a "relation" concept where you can define relationships among two or more tables and iterate through them with related records being brought into the record buffers automatically as the iteration proceeds. Due to its complexity, this feature has **not** been implemented in Python CodeBase Tools.

Iterating through Multiple Tables Simultaneously

Taking scenarios in order of increasing complexity, we provide examples of how to iterate through multiple related tables

One-to-one relationships

In this simplest of situations, you are scanning through records of a parent table and need to gather related information from one or more tables such that there is one record in each related to the current record of the parent table. In the Finding Multiple Records Matching Your Criteria section above, we presented three approaches to iterating through a table gathering selected records: locate(), copytoarray() and seek(). In another section, we introduced the scan() function, which also can operate as an iterator through a table. Each of these may be used as the starting point for multi-table record retrieval.

In a first example, we want to assemble a list of products and their inventory counts for all products stored in warehouse POR. The warehouse information is stored in the PRODCATG table, so we can either iterate through the PRODCATG table for categories stored in the POR warehouse or we can iterate through all records in the PRODUCTS table and interrogate the PRODCATG for each to find those stored in POR.

We begin by scanning through the PRODCATG records where the product category is stored in the POR warehouse. Then search for all instances of that category using the locate() functions to find PRODUCT records in each category. See Figure 19 below:

```
import os
from CodeBaseTools import cbTools
oDB = cbTools()
os.chdir(r"E:\GIT_Repositories\Python-CodeBase-Tools\TestFiles")
# Open both tables
oDB.use("PRODUCTS")
oDB.use("PRODCATG")

xPORProdList = list() # Define the target list to store the results

for xRec in oDB.scan(forExpr="CATG_WHSE='POR'"):
        # Iterate through each PRODCATG record (currently selected table) that is stored in 'POR'
        cCatg = xRec["PROD_CATG"]
        # Look for all products in that category
        oDB.select("PRODUCTS")
        if oDB.locate("PROD_CATG='" + cCatg + "'"):
                xItem = oDB.scatter()
                xPORProdList.append(xItem)
                while oDB.locatecontinue():
                        xItem = oDB.scatter()
                        xPORProdList.append(xItem)
                oDB.locateclear()
                oDB.select("PRODCATG")
# Print out the results.
for xProd in xPORProdList:
                print(xProd["PROD_NAME"], xProd["PROD_COUNT"])

# close the tables
oDB.closetable("PRODUCTS")
oDB.closetable("PRODCATG")
```

Figure 19

The key feature in Figure 19Figure 19 is the repetition of the call to oDB.locatecontinue() such that if it returns True, the current record is retrieved with the scatter() function and stored in the xPORProdList list() created near the top of the example code. Once the oDB.locatecontinue() returns False, the retrieval is done, a locateclear() is called to terminate

the locate() process. Finally an iteration through the xPORProdList is included to print out each found record. (Note that the actual output of this iteration is omitted for space reasons in the example.)

Alternatively, in Figure 20 below we can assemble a list of products using the copytoarray() and use the seek() function for each element of that resulting list to pick out the ones we want from the PRODCATG table where the CATG_WHSE value is "POR.

```
import os
from CodeBaseTools import cbTools
oDB = cbTools()
os.chdir(r"E:\GIT_Repositories\Python-CodeBase-Tools\TestFiles")
# Open both tables
oDB.use("PRODCATG")
oDB.use("PRODUCTS")

xProdList = oDB.copytoarray(alias="PRODUCTS", maxcount=20000)
xPORProdList = list()

oDB.select("PRODCATG")
for xProd in xProdList:  # examine each one in the big list
        # Iterate through each PRODCATG record (currently selected table) that is stored in 'POR'
        cCatg = xProd["PROD_CATG"]
        bFound = oDB.seek(cCatg, "PRODCATG", "PROD_CATG")
        if bFound:
                cWhse = oDB.curvalstr("PRODCATG.CATG_WHSE")
                if cWhse.strip() == "POR":
                        xPORProdList.append(xProd)

# Print out the results.
for xProd in xPORProdList:
                print(xProd["PROD_NAME"], xProd["PROD_COUNT"])

# close the tables
oDB.closetable("PRODUCTS")
oDB.closetable("PRODCATG")
```

Figure 20

In Figure 20 above, note the use of copytoarray() to load all PRODUCTS records into a single list. We obviously don't need all of them, but in this instance it may simply be faster to pull all of them into one list, and then iterate through that in-memory list to look for records where the corresponding PROD_CATG record has a CATG_WHSE value of "POR". The final result is stored in the result list xPORProdList.

Special Considerations

With 4 ways to iterate through a table (copytoarray(), locate(), scan() and index-based seek()), you can combine these in various ways to extract data from multiple tables related in potentially complicated ways. Your choice of tools may depend on some of the following considerations:

- Scan() expressions may not be nested, and copytoarray() may not be used in the scan() either. If you need to examine individual records in a different table during a scan(), you can use the locate() mechanism or the seek() – either with or without index support. [The locate() mechanism makes use of index expressions if possible to gain some speed, but the seek() coupled with an explicit index reference as in Figure 14 is faster.]

- Scan() does allow (but not require) you to use an index to accelerate performance. Timed testing may be needed to find the best indexes to use.
- By default the scan() function returns a full record dictionary for all fields in each record converted to their Python types. In large tables where you may need only a few field values from each record, this causes a lot of unnecessary processing overhead and time. You have two choices in that case: 1) Use the fieldlist parmeter with the fields you'll be needing, or 2) Avoid field loading completely by passing noData=True, in which case each iteration through the scan returns only the record number as an integer, not the full field value dictionary and you may need to use one of the curvalx() functions to test the field values you are interested in.
- A combination of index ordering with an initial seek() and subsequent skip() as illustrated in Figure 14, is generally the fastest, but most verbose tool – and can be used within scans and can be nested for more complicated relationships.
- Look-ups in small tables may be replaced by loading the table contents into a Python dictionary with the table key value as the dictionary key values. Then you can use a simple, fast, in-memory dictionary lookup to obtain the required information. For tables less than a few hundred records and many repeated references to the data, this may well be your highest performing option.
- For more information see the section on Selecting and Finding Records.

Object Oriented Table Handling

For those Pythonistas who prefer pure object-oriented coding in their favorite language, an alternate approach to using the CodeBase Tools is available that provides for accessing most of the features described in this manual using table objects. However, warning, this capability is not being actively maintained, and some features may not behave exactly the same as their procedural equivalents illustrated in this User Guide.

Still, there is clarity and brevity that are made possible by the pure object-oriented versions of these capabilities.

Creating a Table Object

The cbTools() object, the primary starting point for CodeBase Tools has a factory function TableObj() that is used to create table objects. Once a table object has been created, its properties will be the field values for the table, and methods can be called on it like goto() and recno(). An example Idle session is illustrated below in Figure 21 to show how this works:

```
>>> import os
>>> from CodeBaseTools import cbTools
>>> os.chdir(r"E:\GIT_Repositories\Python-CodeBase-Tools\TestFiles")
>>> oDB = cbTools()
>>> oProducts = oDB.TableObj("PRODUCTS.DBF")
>>> oProducts.name
'PRODUCTS'
>>> oProducts.goto("BOTTOM")
True
>>> oProducts.prod_name
'Garden Planter'
>>> oProducts.goto("TOP")
True
>>> oProducts.prod_name
'Test Hardware Product'
>>> oProdCatg = oDB.TableObj("PRODCATG.DBF", readOnly=True)
>>> oProdCatg.recno()
1
>>> oProdCatg.close()
True
>>> oProducts.close()
True
>>> oProducts.goto("TOP")
Traceback (most recent call last):
  File "<pyshell#15>", line 1, in <module>
    oProducts.goto("TOP")
  File "C:\Python312\Lib\site-packages\codebasetools\CodeBaseTools.py", line 5580, in __getattr__
    raise AttributeError( "Closed file")
AttributeError: Closed file
```

Figure 21

In the Figure 21 code above, the table object is created with the TableObj() function to which we pass the fully qualified path name of the physical table the object will represent. (To simplify the code, we first change the current directory to where the table exists, but we could simply pass a fully qualified path name for the target table to the TableObj() constructor.) The object has a .name property to confirm what we have opened. We call the goto() function to position the record pointer at the end of the table, which allows us to reference the content of the last record using simple property values of the table object, as in oProducts.prod_name. [HINT: No effort is made to deal with the case where a standard object property is the same as a field in the table, it is your job as a programmer to avoid this situation, if possible.]

A second table object is created, but this time we pass an additional parameter to make this table read only. The close() function (or closetable()) function of the table object is called to close the table, making the object reference variables orphans. Once the close() function is called, attempting to call a function to control the table (as in the last goto("TOP") attempt in Figure 21) will result in a Python AttributeError.

Table object Operations

Most, but not all, cbTools() functions which relate to the currently selected table may be called from a TableObj instance, but there are special considerations to take into account as discussed below:

- In the table object mode, set deleted status is set to True at creation. You should test the performance of the deleted() assignment if you decide to change that in your use of this approach.

- The property names used to store the field values for the current record are NOT case sensitive. They may also conflict with internally defined property names.
- If there is no current record, the value of all properties will be None. Since None may be a valid value for a field, you should check the value of the cErrorMessage property to determine if there really is no record there.
- In the table object mode, as with the default mode, you can open a table multiple times with different alias values. Each table behaves separately relative to current record and index settings. However, you MUST specify a unique alias value in the TableObj() function call.

  Example:

  ```
  oMyTable = oCBT.TableObj(r"c:\\somepath\\sometable.dbf", alias="MYNEWALIAS")
  ```

- Some cbTools() functions like flush() and locate() may not work as expected. Test, Test, Test.
- Deleting the variable containing the table object name or setting its value to None does NOT close the table. You MUST remember to issue a close() or closetable() method to force the table closed before destroying the object. This must be done for each of any multiple instances of the table object you have created.
- The isexclusive() method may report incorrect results if called on one of these objects but the exclusive=True parameter works when passed to the TableObj() constructor.
- You can mix the object and default versions in the same code module (but do so carefully and test carefully).
- Calling any method on the table object will set the currently selected table to the table you are referencing. If you are using a mix of the table object approach and the default approach, be aware that the "currently selected" table may not be what you expect after any of the table objects are accessed.
- Finally, test your applications thoroughly if you are using the TableObj() mode, to make sure that all functions you are using work correctly in this mode. We make no guarantees. [HINT: Refining the object-oriented table handling might be a good project for someone wanting to contribute a capability to this open-source project!)

Special Topics

Using Temporary Indexes for Special Processing

Every index tag added to a table's .CDX adds both increased analytical power and processing time for updates and changes to the table. Sometimes it may be useful to have an index that

you just need briefly for one bit of processing, but don't need the rest of the time and don't want to consume processing time maintaining it when not needed. Enter the temporary index feature of CodeBase Tools.

The maketempindex() and selecttempindex() functions provide this feature. The example below in Figure 22 shows how they work. We first open the PRODUCTS.DBF table and call the maketempindex() function. We define a complex index expression involving 3 fields. There is no current index like this which produces a sequence by category name and within that alphabetically by product name. The value returned by the function is 0, which is a valid temporary index ID (A value of -1 indicates an error – usually because the expression parameter is not valid.) We then use that return value as the parameter for selecttempindex() which automatically selects our temporary index as the primary one for scanning the table. Running an iteration through the scan() function results in the ordering of records as shown in Figure 22**Error! Reference source not found.**:

```
>>> from CodeBaseTools import cbTools
>>> oDB = cbTools()
>>> os.chdir(r"E:\GIT_Repositories\Python-CodeBase-Tools\TestFiles")
>>> oDB.use("PRODUCTS.DBF")
    True
>>> nTempIndex = oDB.maketempindex("UPPER(PROD_CATG) + UPPER(PROD_NAME) + STR(PROD_COUNT)")
>>> nTempIndex
    0
>>> oDB.selecttempindex(nTempIndex)
    True
>>> for xRec in oDB.scan():
...     print(xRec["PROD_CATG"], xRec["PROD_NAME"])
...
...

    GAR Another Garden Product
    GAR Garden Planter
    GAR Test Garden Product
    HAR Test Hardware Product
    HOM Test Home Appliance - Refrigerator
>>> oDB.closetable("PRODUCTS")
    True
```

Figure 22

We can then close the table. This automatically closes the temporary index and removes it from the disk. You need take no further action. Some basic rules for temporary indexes:

- The maketempindex() function applies to the currently selected table. It is "attached" to that table's alias name. If the same table is open elsewhere in the program with a different alias, that instance of the table with the other alias has no access to the temporary index.
- You can create multiple temporary indexes on the same table, but only one can be active for iterating through the table at one time as determined by the selecttempindex() function.
- The active temporary index specified by the selecttempindex() function will be updated automatically if changes to the table itself are made while it is active.

- You have no access to the physical file containing the index. The temporary index is created and removed automatically when you are finished with it and the associated table is closed.
- There is a tagFilter parameter for the maketempindex() function that allows you to restrict the index from seeing any records which do not meet the logical expression in the tagFilter parameter.
- The temporary index applies to the skip(), goto(), and scan() functions. Also it respects the setdeleted() setting while scanning through the table.

Exporting to and Importing from XML

Two complementary functions are provided to output data to XML and to retrieve XML in the same format back into an appropriately structured DBF table. These functions provide the most commonly used subset of capabilities in corresponding functions in Visual FoxPro.

cursortoxml()

In VFP the CURSORTOXML() function is a powerful means of copying all or part of a "cursor" or open DBF table into XML form in several different ways. This method provides a subset of that functionality, creating a file that is identical to the default VFP output where each record has a tag, and each field in that record has its tag and the corresponding value contained in the tag. The top-level tag name is always <VFPData> unless you specify otherwise. All field and cursor record tags are lower case. The function outputs fields in each record in the order in which they appear in the table or in the order in the cFields list. Function parameters are:

- **cAlias**: Pass the alias name if not the currently selected table.
- **cForExpr**: Pass a valid logical expression to select records. If omitted, all records will be output.
- **cFields**: Pass a comma delimited list of field names to limit the output to selected fields.
- **bStripBlanks**: By default, trailing blanks on text fields are left in place. Set this True to trim them.
- **cFileName**: Pass a file name to output the result directly into that text file. If omitted, then the method will return the resulting XML as a string.
- **cpRecName**: Each record output will be set off in a repeating tag name. If a value is provided for this parameter, it will be used for that repeating tag name, otherwise the system will generate a single unique 8-character name in all lower case for the record tags.
- **cAltMainTag**: By default this function follows the VFP practice and wraps everything in <VFPData> tags. if you want something different, specify the base name here. The system will add the brackets.
- **Return value**: The XML text as a string unless the cFileName parm is not empty. If cFileName is "", and if an error occurs, the function will return "". If cFileName is a file

name, returns True on success – writing the XML into the file by that name, False on failure, and sets cErrorMessage.

NOTE: The XML text is stored in memory until output at the end. Very large tables may cause memory issues as a result, especially in 32-bit versions of Python.

xmltocursor()

In VFP the XMLTOCURSOR() function is the converse of the CURSORTOXML() function. It makes possible the loading of XML created by CURSORTOXML() or other similar tools back into a cursor. The xml data will be stored back into a table (appending the records) which has some or all of the fields specified in the XML data. Warning: it will be up to you to make sure that if the receiving table has primary keys, which must be set externally to unique values, that those will be handled somehow, either by prepopulating those fields in the XML or using an intermediate table with no indexes to store and process the key field values.

No capability is provided to create a table into which to load the XML based on the XML data, so you must first create a table to receive the XML records, and have at least one field name which corresponds to a field in the XML. Attempting to load XML into a DBF table with no matching field names results in an error condition.

The primary tag name for the XML data is typically <VFPData> (see cursortoxml() above), but may be something else. Tag names for individual records may be any value. Tag names for fields must match the spelling (but not the case) of fields in the target table, or the data element will be ignored.

Function parameters are:

- **cAlias**: Alias of an open table into which to load the data from the XML. If blank, the currently selected table will be used.
- **cXML**: Text string with the XML content to be loaded. Leave "" if loading from a file.
- **cFileName**: File name from which to load the XML. Leave "" if cXML is passed.
- **Return Value**: Number of records loaded. 0 could indicate an error.

See Figure 23 below for an example of XML text output for the PRODUCTS.DBF table:

Example XML Output – Default Parameter Values for Products.DBF

```
<?xml version = "1.0" encoding="UTF-8" standalone="yes"?>
<VFPData xml:space="preserve">
        <ujohyool>
                <prod_key>11000</prod_key>
                <prod_code>ABDCode1            </prod_code>
                <prod_name>Test Hardware Product                              </prod_name>
                <prod_catg>HAR</prod_catg>
                <prod_count>100.0</prod_count>
        </ujohyool>
        <ujohyool>
                <prod_key>11001</prod_key>
                <prod_code>DFG_CODE            </prod_code>
                <prod_name>Test Garden Product                                </prod_name>
                <prod_catg>GAR</prod_catg>
                <prod_count>50.0</prod_count>
        </ujohyool>
        <ujohyool>
                <prod_key>11002</prod_key>
                <prod_code>HOMEAPP1            </prod_code>
                <prod_name>Test Home Appliance - Refrigerator                 </prod_name>
                <prod_catg>HOM</prod_catg>
                <prod_count>3.0</prod_count>
        </ujohyool>
        <ujohyool>
                <prod_key>11003</prod_key>
                <prod_code>DFG_THING           </prod_code>
                <prod_name>Another Garden Product                             </prod_name>
                <prod_catg>GAR</prod_catg>
                <prod_count>30.0</prod_count>
        </ujohyool>
        <ujohyool>
                <prod_key>11004</prod_key>
                <prod_code>DFG_PLANTER         </prod_code>
                <prod_name>Garden Planter                                     </prod_name>
                <prod_catg>GAR</prod_catg>
                <prod_count>5.0</prod_count>
        </ujohyool>
</VFPData>
```

Figure 23

Exporting to and Importing from Excel Format

Excel and Python CodeBase Tools each have their own domain of usefulness. Excel offers computational tools and analytical traceability. The DBF data storage platform allows for live user interaction (including multiple users simultaneously), large data sets, and enforcement of pre-designed data structures. Many operational and analytical systems benefit by having programmatic conversion of DBF data to Excel format and vice versa. Python CodeBase Tools provide this capability using a reasonably priced commercial product that enables production-quality data conversion at speeds much greater than the default “automation” features built into Microsoft Excel itself. Everything is provided for you in the Python CodeBase Tools installation except the license key, which you can obtain by purchase from the LibXL.com vendor website.

NOTE: The proper functioning of the DBF to XLS and reverse functions requires that ctypes be installed in your Python system. Also, these modules are designed for use in Python 2.x and 3.x interchangeably. All **32-bit** versions of Python 3.x supported for CodeBase Tools are supported for these capabilities. Versions of Python 2.x before 2.7 may not work correctly.

Using DBFXLStools2 in Your Programs

The heart of this feature is the DBFXLStools2.py module and the class definition contained in it: DbfXlsEngine(). This component is used both for export to and import from Excel. See below in Figure 24 for an example of a simple export from DBF to .XLSX format:

```
>>> from CodeBaseTools import cbTools
>>> from DBFXLStools2 import DbfXlsEngine
>>> oDB = cbTools()
>>> with DbfXlsEngine(oVFP=oDB) as oXL:
...     bResult = oXL.dbf2excel("PRODUCTS.DBF", "PRODUCTS.XLSX")
...     print(bResult)
...
True
```

Figure 24

The code in Figure 24 first creates an instance of cbTools(), which will be used to open and extract data from the source DBF table. Then an instance of the DbfXlsEngine() object is created using the "with" construct with a reference of oXL. This constructor is passed the object reference to cbTools in its oVFP parameter. Then the oXL class function dbf2excel() is called using our example table PRODUCTS.DBF. Note that the second parameter is the name of the output Excel spreadsheet. In this case we have been explicit for the output to be in .XLSX format – the latest Excel file version. If your application needs the earlier .XLS format, simply giving the output file name that extension will yield that result (but .XLS spreadsheets are limited to 65000 rows, so be alert to that and other XLS limitations).

The screen shot from Excel below shows the output of that simple function call:

	A	B	C	D	E
1	Created by LibXL trial version. Please buy the LibXL full version for removing this message.				
2	11000	ABDCode1	Test Hardware Product	HAR	100
3	11001	DFG_CODE	Test Garden Product	GAR	50
4	11002	HOMEAPP1	Test Home Appliance - Refrigerator	HOM	3
5	11003	DFG_THING	Another Garden Product	GAR	30
6	11004	DFG_PLANTER	Garden Planter	GAR	5

Figure 25

In Figure 25 the spreadsheet has the trial version notice as it was produced without the commercial license information code from the old version of the libxl.dll file distributed in this repository. *That trial version notice conceals the row 1 information in which each column header contains the upper-case field name appearing in that column.* The trial version also has a size limit on the output spreadsheet. There is a license text file supplied in the installation directory, into which you can paste your paid-for license to remove the trial notice.

For more sophisticated output formatting, other named parameters are available in the dbf2excel() function to customize your conversion output:

- **headerlist**: A Python list() object containing a list of the column headers to be output with the field columns. If the list contains fewer items than columns being output, the remainder of the columns will have the field names as column headings. If this value is None or not passed, all column headers will be field names.
- **titleone**: Text string which will appear in its own row at the top of the spreadsheet in a Title sized font. Default is an empty string, in which case there is no Top Title row added.
- **titletwo**: Text string which will appear in its own row below the Top Title from titleone. Default is an empty string or no titletwo output.
- **hiddenNames**: A Boolean which tells the function to insert a hidden row directly below the column header row containing the field names for each of the columns. You can set this to true if you have set up "human readable" column heading names (possibly internationalized) in the headerlist parameter and need for an automated spreadsheet reading process to be able to identify what data is in each column.
- **Noclose**: A Boolean which tells the function to leave the source table open in cbTools() for further processing.
- **fieldList**: A text string containing a comma-delimited list of field names to be included in the output. Names not found will be skipped. If no valid field names are found, an error will be raised.
- **cTagName**: the name of the index tab associated with the table in which order to output the rows of the spreadsheet.

The excel2dbf() function is a powerful tool for converting an Excel spreadsheet to a DBF table. In its simplest form, it converts the spreadsheet and loads the data into an existing DBF table. However, if no .DBF table exists to receive the data, the function will inspect the spreadsheet columns and do its best to create a .DBF table with appropriate field types into which the spreadsheet can be imported.

The excel2dbf() function is executed within a "with" construct. It has a number of optional parameters which control how it handles the output .DBF table:

- **lcExcelName**: Text string containing the fully qualified path name of the source .XLS or .XLSX spreadsheet to convert.
- **lcDbfName**: Text string containing the fully qualified path name of the output .DBF table to be the target of the conversion.
- **sheetname**: Text string with the desired worksheet name from the spreadsheet from which to extract the data into the .DBF table. If empty, then the first worksheet in the spreadsheet will supply the data.

- **appendflag**: Boolean value. If True, then any existing content in the .DBF table will be retained and the data from the spreadsheet will be appended into the table after the existing data. If False, then any data in the DBF table will be cleared, if any, and the spreadsheet data will replace it.
- **templatedbf**: If the name of a valid .DBF table is provided in this parameter, then the structure of that table will be used to create a new table into which the contents of the spreadsheet will be written. If the named .DBF output table already exists, it will be overwritten, and the appendflag parameter will be ignored.
- **fieldlist**: A Python list() object containing the names of fields in the source speadsheet (using the column headings to identify field name sources) to be output into the .DBF table. In this case, the first row of the sheet must contain the field names. If any of the names is invalid in the .DBF format (or empty), an error will be raised.
- **showErrors**: A Boolean, which, if True, tells the conversion engine to print to stdout a message explaining error conditions found during processing. No stdout output will be produced if the value is False.
- **dateFormat**: A text string providing the recommended conversion interpretation of text string dates found in the spreadsheet. If the date value in a cell is a valid Excel date/time type, it will be converted automatically, but dates may be found in string/char format and the conversion rule will be needed in that case. The default is 'XXXX' in which case any text date value will be interpreted as YYYYMMDD, if valid. Other values allowed are MDY (for mm/dd/yy), MDYY (for mm/dd/yyyy), DMYY (for dd/mm/yyyy) and DMY (for dd/mm/yy).
- **Return Value**: will be a numeric value indicating how many records were loaded into the .DBF table. A value of -1 indicates an error. However, if a 0 is returned, you may want to investigate the causes if there is data in the spreadsheet. The cErrorMessage property will contain a message indicating what went wrong.

Note that if a .DBF output file name is provided which does not appear on disk and if there is no templatedbf value given, then the process will attempt to determine the appropriate .DBF field type for each column, will take the field names from the column header row (if possible) and will build a .DBF table accordingly. The result should be validated by further inspection.

Using ExcelTools.py

The two key functions in DBFXLStools2 and DbfXlsEngine() depend on a powerful wrapper around the LibXL.DLL product called ExcelTools.py, provided with this repository. Numerous functions are built into this wrapper module for defining formats and applying them to cells, handling multiple cells, clearing ranges, adding graphics, spanning rows and columns, adding formulas, and so on. These can be accessed directly by your applications for sophisticated

manipulation and formatting of Excel spreadsheets. Detailed documentation of this module is beyond this introductory User Guide, and we recommend that you review the doc strings in it for details on how to use its features. You can find many examples of how they work in the DBFXLStools2.py module as well. **This module makes use of Python ctypes and requires a 32-bit version of Python, either version 2.7 or 3.x**.

Calculation and Summarization Tools

Elaborate calculation and summarization of modest data sets is usually best handled in Excel, with its vast array of computational functions. But CodeBase Tools offers a couple of utility functions that can do several basic calculation chores. The first is countfor(), which is illustrated in Figure 7, and simply counts the number of records for which a logical expression returns True.

For more complex accumulation we have calcfor() which always applies to the currently selected table and returns one of four kinds of computations on a set of records meeting your specified logical expression. This is illustrated below:

```
>>> import os
>>> from CodeBaseTools import cbTools
>>> os.chdir(r"E:\GIT_Repositories\Python-CodeBase-Tools\TestFiles")
>>> oDB = cbTools()
>>> oDB.use("PRODUCTS.DBF")
True
>>> oDB.calcfor("SUM", "PROD_COUNT")
188.0
>>> oDB.calcfor("AVG", "PROD_COUNT")
37.6
>>> oDB.calcfor("SUM", "PROD_COUNT", "PROD_CATG='GAR'")
85.0
```

Figure 26

In Figure 26 we open the PRODUCTS.DBF table and immediately call the calcfor() functions as illustration. Note that immediately after the use() function, that table is the currently selected one. The optional calculation codes are “SUM”, “AVG”, “MAX”, and “MIN”. Here we see the AVG and SUM codes used on the PROD_COUNT field. In the third example function call, we also add a conditional expression so that the PROD_COUNT field is only summed for PRODUCTS.DBF records where the PROD_CATG field is “GAR”.

Note that the PROD_CATG field is 10 characters in width, but the ‘=’ in the logical expression only attempts to match as many characters as are found in the right-hand part of the expression, ignoring anything beyond the “GAR”. If you had chosen to set the logical symbol as ‘==’, you would have needed to match the full 10-character value of “GAR “.

Be careful that you specify the second parameter as the name of a numeric field. Running the calcfor() function on a character or other non-numeric field will always return a value of 0.0.

Also, this function should never be called in the middle of a locate() series, but it may be called on the current table in a scan() section.

Special Considerations for Storing Python Strings

The biggest change as Python migrated from 2.x to 3.x versions was the handling of string variables. Python 2.x had a str() type which stored essentially ASCII data and a Unicode() type which stored the string in a Unicode form. With Python 3.x, all strings are stored as a special Python version of Unicode as a 'str' type and non-linguistic strings are stored as 'bytes' or 'bytestring' types. Since CodeBase Tools handles both Python 2.x and 3.x, special handling was added back in 2018 to accommodate these differences when storing Python data into .DBF table fields.

The DBF format supports 4 kinds of character-oriented fields: char, memo, char-binary, and memo-binary. In general, Python 2.x strings are mapped into char or memo type fields and Python 2.x Unicode values are mapped into char-binary or memo-binary fields. ('char' fields are string fields with a defined length – maximum of 255 bytes. 'memo' fields are string fields with a nominally unlimited number of characters, see page 54. 'char' fields are automatically padded with blanks – character 32 values – to their full nominal width.)

Detailed rules for how these field types are handled in each Python version are given below, along with an explanation of how codecs are mapped to DBF Code Page values.

Python 2.7 (earlier 2.x versions are not officially supported)

Default data storage is 8-bit strings as found in Python str objects. Embedded nulls will be treated as end of string markers and will not be stored. Unicode strings will be coerced to Code Page 1252 before saving.

Applies to scatter, gather, insert, replace, and curval type functions. Note that each of these functions has a 'coding' parameter, defaulting to 'XX', which can be used to control data conversions when data is written into or read from these fields. The first 'X' character of the code indicates how ordinary char and memo fields are translated. The second 'X' character applies to the binary forms of char and memo fields.

If bytearray types (with or without embedded nulls) are supplied for these fields, they will be copied into the target field as pure binary data. In the case of char fields, any unused space to the right of the data will be padded with blanks. Memo fields will contain only the data content. If you write binary data from bytearray objects into these standard text fields, be sure that you handle the content accordingly when reading it back. Also beware of FoxPro Code Page translations to which these fields are subject by default when the table is read from or written to by Visual FoxPro (Code Page translations are NOT performed by CodeBase Tools.).

The following special values for the coding parameter will be allowed optionally in these functions for str and unicode object data sources.:

For Char and Memo:

- ' ', 'X', or None = The default storage and retrieval.
- 'W' = Windows Double Byte Unicode encoding for the specified Windows Code Page (default Code Page in the U.S.A. is usually cp1252) on the local machine. Source data may be string or bytearray, and conversion will be performed. Unicode will be coerced to the default Code Page.
- '8' = UTF-8 encoding. Source may be string, bytearray, or Unicode.

For Char(binary) and Memo(binary):

Default data storage is a non-text sequence of 8-bit bytes with no textual meaning. The raw contents of strings will be stored, but with strings terminated when a NULL is encountered. Bytearrays will be copied character for character, regardless of the presence of null bytes.

The following special coding will be allowed optionally in these functions with string or unicode type data source objects:

- ' ', 'X', or None = The default storage and retrieval
- 'D' = Windows Double Byte Unicode encoding for the specified Code Page (default Code Page in the U.S.A. is usually cp1252) on the local machine.
- 'U' = UTF-8 encoding.
- 'C' = Custom Code Page code. See Python codecs documentation for allowable values. Examples include: "cp1252" - Western Europe, "cp1251" - Cyrillic (Russian, etc.), "gb2312" - Simplified Chinese.

Python 3.x (3.6 and higher)

All 3.x Python strings are Unicode. This module will attempt to code Python strings for storage in the field type specified. If the Unicode string value cannot be converted into a form suitable for the field type, an error will be triggered. Applies to scatter, gather, insert, replace, and curval type functions.

By default, conversion of the strings to and from 8-bit Code Page 1252 will be attempted for non-binary char and memo fields. This will handle most special characters for Western European languages.

Version 3.x also has a bytes and bytearray type (differences are very subtle, but the both contain sequences of bytes that don't necessarily have any textual meaning). Both of these binary data types may be stored in both plain and binary char/memo fields, but you will need to be careful with their data retrieval. In all such cases, the contents of the bytes and bytearray objects will be stored byte for byte into the target field.

The following special coding will be allowed optionally in these functions:

Char and Memo:

- ' ', 'X', or None = The default storage and retrieval (cp1252, 8-bit coding or plain ASCII)
- 'W' = Windows Double Byte Unicode encoding for the specified Code Page (default Code Page in the U.S.A. is usually cp1252) on the local machine.
- '8' = UTF-8 encoding.

Char(binary) and Memo(binary):

Default data storage is a non-text sequence of 8-bit bytes with no textual meaning.

The following special coding will be allowed optionally in these functions:

- ' ', 'X', or None = The default storage and retrieval. For Bytes and ByteArray data, this will be a simple copy of the binary data retrieved into a Python bytes() value. Char fields will be filled out with nulls. String (Unicode) data will be stored as Code Page 1252 and padded out with blanks. (For pure binary data, memo (binary) should be preferred.)
- 'D' = Windows Double Byte Unicode encoding for the specified Code Page (default Code Page in the U.S.A. is usually cp1252) on the local machine.
- 'U' = UTF-8 encoding.
- 'C' = Custom Code Page code. See Python codecs documentation for allowable values. Examples include: "cp1252" - Western Europe, "cp1251" - Cyrillic (Russian, etc.), "gb2312" - Simplified Chinese. There are many others.

General Notes

Note that with all the non-default codings, you are expected to know what codings are contained in the fields you are reading and writing. Data which cannot be transformed by the Python codecs manager will trigger a Python error and will need to be trapped by a try... except... block.

Special consideration for systems integrated with Visual FoxPro applications: If you specify a custom coding and are sharing the data with Visual FoxPro, you are responsible for ensuring the codings match on both sides. VFP will dynamically change field data in Char and Memo fields (but not their binary versions) if the table and local machine Code Pages do not match. This could result in badly corrupted data, as CodeBase Tools does no automatic Code Page conversions.

Also, be careful with functions that read or write data from and to multiple fields. In such cases, the custom coding specifiers shown above will be applied to every field of the

appropriate type. If that is not what you want, process fields individually using replace() or curvalstrex().

The reading and writing functions in CodeBase Tools have default values of "XX" for the coding parameter. In most instances where you are working with European languages that will work just fine. In cases where you are working with double-byte Unicode languages like Mandarin and Japanese, we recommend that you use the binary versions of the char and memo fields. In that case you'll need to set the widths of the fields to 2X the maximum number of characters, and you can set the coding parm to either 'XX' or '88'. Use of Windows Code Pages intended for multi-byte character languages are supported, but we recommend you test carefully with sample text data when using the coding value of 'W' or 'WW'.

Large Mode Implementation and Features

By design, the default mode of CodeBase Tools is limited by the same constraints as the Visual FoxPro system, which it is intended to interoperate with. But as an application that is available for both 32-bit and 64-bit environments, CodeBase natively is not constrained by VFP limitations. Thus, there is both a normal (default) mode and a "Large Mode" implementation available in CodeBase Tools. The factory constructor cbTools() when called without any parameters, produces a CodeBase Tools object using the VFP compatibility mode as in all examples above. To obtain a CodeBase Tools object running with Large Mode, simply pass a True as the one parameter as shown in the example below in Figure 27:

```
>>> from CodeBaseTools import cbTools
>>> oDB = cbTools(True)
>>> oDB.islargemode()
True
```

Figure 27

Some important considerations. Multiple instances of CodeBase Tools objects with different settings for Large Mode as True and False are not recommended. Your application should use one or the other, but not both in the same module. Also, if you are intending to interoperate with Visual FoxPro or other compatible systems, you will be able to read and write DBF files that comply with VFP constraints with either mode, but any attempt to write to a DBF file to make it larger than VFP capacity will result in VFP regarding it as corrupt and unable to open it. For comparison between VFP compatibility constraints and the Large Mode constraints, see the section on Capacities and Limits in the Appendix on page 54.

Appendixes

Expressions Allowed in Indexes and Searches

Arithmetic operators and expressions are allowed in index tag definitions and searches in scan() and locate() functions, among others. The details of exact arithmetic operator syntax are provided in the individual language Users Guides published by Sequiter and downloadable from the GitHub repository for CodeBase-for-DBF. In general these operators will work as you expect. The one key difference is that some logical constants and relational operators have different formats required in expressions defined for CodeBase Tools functions, as shown in the figure below:

Operator or Constant Name	Symbol in CodeBase Expression
And	.AND.
Or	.OR.
Equal To (evaluated to length of comparison string)	=
Exactly Equal To	==
True	.T.
False	.F.
Not	.NOT.
Not Equal	<>
Less Than	<
Less Than or Equal To	<=
Greater Than	>
Greater Than or Equal To	>=
Addition or Concatenation	+
Subtraction	-
Contains	$

Figure 28

```
oDB.use("PRODUCTS.DBF")
True
oDB.calcfor("SUM", "PROD_COUNT", "PROD_COUNT<>0 .AND. PROD_COUNT>5 .AND. PROD_CATG='GAR' .AND. 'TEST' $ UPPER(PROD_NAME)")
50.0
```

Figure 29

Examples for the use of .AND., and $, plus the use of the function UPPER() are illustrated in Figure 29 above.

CodeBase Tools provides some 32 functions which may be embedded in Index Tag expressions, and used in selecting records via the locate(), calcfor(), countfor(), scan(), and cursortoxml(). Most, but not all, of these are compatible with Visual FoxPro when they are incorporated into Index Tag expressions. The table following provides a detailed list of these functions (note comments for those not compatible for Visual FoxPro indexes). Collectively, the constants and

operators above and the embeddable functions below provide a rich capability for ordering and selecting records in CodeBase Tools.

Embeddable Functions for Index Tags and Record Selection in Python CodeBase Tools

ALLTRIM(CHAR_VALUE)

This function trims all of the blanks from both the beginning and the end of the expression.

ASCEND(VALUE)

This function is not supported by dBASE, FoxPro or Clipper.

ASCEND() accepts all types of parameters, except complex numeric expressions. ASCEND() converts all types into a Character type in ascending order. In the case of numeric types, the conversion is done so that the sorting will work correctly even if negative values are present.

CHR(Integer_Value)

This function returns the character whose numeric ASCII code is identical to the given integer. The integer must be between 0 and 254.

Example: CHR(65) returns A.

CTOD(Char_Value)

The character to date function converts a character value into a date value.

e.g. " CTOD("11/30/88") "

The format of the resulting character value is specified by the current date format string which is by default "MM/DD/YY". To set the default to other formats use the setdateformat() function in CodeBase Tools.

DATE()

The system date is returned.

DATETIME(Year, Month, Day [, Hour [, Minute [, Second]]])

The given date and time is returned as a DateTime.

e.g. "STOPTIME = DATETIME(2003,03,21,13,30,00)"

This expression would evaluate to .TRUE. if the DateTime field STOPTIME is set to March 21, 2003 at 1:30 pm.

DAY(Date_Value)

Returns the day of month of the date parameter as a numeric value from "1" to "31".

e.g. "DAY(DATE())"

Returns "30" if it is the thirtieth of the month.

DESCEND(VALUE)

This function is not supported by dBASE or FoxPro. DESCEND() is compatible with Clipper, only if the parameter is a Character type.

DESCEND() accepts any type of parameter, except complex numeric expressions. DESCEND() converts all types into a character type in descending order.

For example, the following expression would produce a reverse order sort on the field ORD_DATE followed by normal sub-sort on COMPANY.

e.g. DESCEND(ORD_DATE) + COMPANY

See also ASCEND().

DEL()

Creates a 1 character string containing the contents of the deleted flag for the current record. Note that this differs from DELETED() which returns a true/false flag whether or not the record is deleted. An asterisk character '*' is used in the datafile to indicate a record is deleted.

This facilitates combining a deleted flag test with other values in an index or composite expression. **Not compatible with Index Tags in Visual FoxPro.**

DELETED()

Returns .TRUE. if the current record is marked for deletion.

DTOC(Date_Value)

DTOC(Date_Value, 1)

The date to character function converts a date value into a character value. The format of the resulting character value is specified by the current date format string which is by default "MM/DD/YY". To set the default to other formats use the setdateformat() function in CodeBase Tools.

e.g. " DTOC(DATE()) "

Returns the character value "05/30/87" if the date is May 30, 1987.

If the optional second argument is used, the result will be identical to the dBASE expression function *DTOS*.

For example, DTOC(DATE(), 1) will return "19940731" if the date is July 31, 1994.

DTOS(Date_Value)

The date to string function converts a date value into a character value. The format of the resulting character value is "CCYYMMDD".

e.g. " DTOS(DATE()) "

Returns the character value "19870530" if the date is May 30, 1987.

Not compatible with Visual FoxPro in Index Tags. Use DTOC instead.

EMPTY(field or value)

Returns TRUE if the field or value is empty. Returns false if it contains a value:

- characters - returns true if the result contains only blanks
- numbers - returns true if the value is 0
- logicals - returns true if the value is false
- dates/datetimes - returns true if the value resolves to zero (field left blank)

IIF(Log_Value, True_Result, False_Result)

If 'Log_Value' is .TRUE. then IIF returns the 'True_Result' value. Otherwise, IIF returns the 'False_Result' value. Both True_Result and False_Result must be the same length and type. Otherwise, an error results.

e.g. "IIF(VALUE < 0, "Less than zero ", "Greater than zero")"

e.g. "IIF(NAME = "John", "The name is John", "Not John ")"

LEFT(Char_Value, Num_Chars)

This function returns a specified number of characters from a character expression, beginning at the first character on the left. The parameter 'NUM_CHARS' must be constant.

e.g. "LEFT('SEQUITER', 3)" returns "SEQ".

The same result could be achieved with "SUBSTR('SEQUITER', 1, 3)".

LTRIM(Char_Value)

This function trims any blanks from the beginning of the expression.

MONTH(Date_Value)

Returns the month of the date parameter as a numeric.

e.g. " MONTH(DT_FIELD) "

Returns 12 if the date field's month is December.

PADL(string, length)

Creates a new string using the input string and the constant numerical length to pad out the string on the left. The final length of the string is the input constant numerical length. Examples:

- PADL("ABC", 4) creates the string " ABC" - 1 blank padded on the left
- PADL("ABCD", 3) creates the string "ABC" - trims due to input length

- PADL(TRIM("ABC "), 5) where the trimmed string contains 3 blank trailing characters creates the string " ABC" - 2 blanks padded on the left

PADR(string, length)

Creates a new string using the input string and the constant numerical length to pad out the string on the right. The final length of the string is the input constant numerical length. Examples:

- PADR("ABC", 4) creates the string "ABC " - 1 blank padded on the right
- PADR("ABCD", 3) creates the string "ABC" - trims due to input length
- PADR(LTRIM(" ABC"), 5) where the trimmed string contains 3 blank preceding characters creates the string "ABC " - 2 blanks padded on the right

RECCOUNT()

The record count function returns the total number of records in the database:

e.g. " RECCOUNT() "

Returns 10 if there are ten records in the database.

RECNO()

The record number function returns the record number of the current record.

RIGHT(Char_Value, Num_Chars)

This function returns a specified number of characters from the end of a character expression. The parameter 'NUM_CHARS' must be a constant.

e.g. "RIGHT('SEQUITER', 3)" returns "TER".

SPACE(numSpaces)

Creates a string containing the input numerical number of blanks. For example SPACE(3) creates a string containing 3 blank spaces (" ").

STOD(Char_Value)

Not compatible with Visual FoxPro indexes.

The string to date function converts a character value into a date value:

e.g. " STOD('19881130') "

The character representation is in the format "CCYYMMDD".

STR(Number, Length, Decimals)

The string function converts a numeric value into a character value. "Length" is the number of characters in the new string, including the decimal point. "Decimals" is the number of decimal places desired. The parameters 'LENGTH' and 'DECIMALS' must be constant. If the number is too big for the allotted space, *'s will be returned.

e.g. " STR(5.7, 4, 2) " returns " '5.70' "

The number 5.7 is converted to a string of length 4. In addition, there will be 2 decimal places.

e.g. " STR(5.7, 3, 2) " returns " '***' "

The number 5.7 cannot fit into a string of length 3 if it is to have 2 decimal places. Consequently, *'s are filled in.

If no length or decimals are specified, the length will be 10 characters padded at left with blanks, and no digits to the right of the decimal point will be displayed.

STRZERO(value, length, decimals)

Identical to STR() except any blanks to the left of the number are replaced by zeros.

Creates a string containing the input numerical value formatted to be of the input numerical length and the optional input numerical decimals. If the optional decimals parameter is left out, the default of no decimals is used. **(Cannot be used in index tags requiring Visual FoxPro compatibility!)**

Examples:

- STRZERO(100.03, 10) creates string "0000000100"
- STRZERO(100, 2) causes an overflow and creates string "**"
- STRZERO(.03, 4) creates the string "0000" since no decimals are specified
- STRZERO(.0004, 3) creates the string “000” since no decimals are specified
- STRZERO(1.44, 2) truncates to "01" because no decimals are specified
- STRZERO(100.03, 10, 4) creates string "00100.0300"
- STRZERO(.0004, 3, 2) truncates to the string ".00"
- STRZERO(0, 6, 3) creates the string "00.000"

SUBSTR(Char_Value, Start_Position, Num_Chars)

A substring of the Character value is returned. The substring will be 'NUM_CHARS' long, and will start at the 'START_POSITION' character of 'CHAR_VALUE'. The parameters 'START_POSITION' and 'NUM_CHARS' must be constant.

e.g. " SUBSTR("ABCDE", 2, 3)" returns " 'BCD' "

e.g. "SUBSTR("Mr. Smith", 5, 1)" returns " 'S' "

TIME()	The time function returns the system time as a character representation. It uses the following format: HH:MM:SS. e.g. " TIME() " returns " 12:00:00 " if it is noon. e.g. " TIME() " returns " 13:30:00 " if it is one thirty PM.
TRIM(Char_Value)	This function trims any blanks off the end of the expression.
UPPER(Char_Value)	A character string is converted to uppercase and the result is returned.
VAL(Char_Value)	The value function converts a character value to a numeric value. e.g. VAL('10') returns "10". e.g. VAL('-8.7') returns "-8.7".
YEAR(Date_Value)	Returns the year of the date parameter as a numeric: e.g. "YEAR(STOD('19920830')) " returns " 1992"

Figure 30

Capacities and Limits

The section above on Large Mode Implementation page 47 explains that there is a “default Mode” and a “Large Mode” of operation of CodeBase Tools. Generally speaking, Visual FoxPro applications (and typically Clipper and dBaseIV applications as well) have limits lower than the limits of default Mode and very much lower than Large Mode. The following table provides specifics of operational size limits for data table and index elements under “default Mode” and “Large Mode” with exceptions for compatibility with Visual FoxPro and other systems indicated.

Data Table Element	Default Mode Limits	Large Mode Limits(*)
Data Table File Size	1,000,000,000 Bytes	8,589,934,590 GB
.CDX Index File Size	4 GB **2 GB for Visual FoxPro Compatibility**	2048 GB with 512-byte index blocks and large file support
.FPT Memo File Size	4 GB	131,072 GB
Memo Entry Size	4,294,967,196 bytes	4,294,967,196 bytes
Maximum Field Width	65517 bytes **254 for Visual FoxPro or dBASE Compatibility**	65517 bytes

Maximum Floating Point Field Width	19 bytes	19 bytes
Maximum Numeric Field Width	20 bytes (Visual FoxPro and dBASE IV Compatibility)	20 bytes
Maximum Number of Fields	2046 **255 for Visual FoxPro Compatibility** **128 for dBASE IV compatibility.**	2046
Maximum Record Width	2 GB **65500 bytes for Visual FoxPro, dBASE IV, or Clipper Compatibility.**	2 GB
Number of Tags per Index	Unlimited	Unlimited
Index Tag Expression Size	240 bytes	240 bytes
Number of Open Files	The number of open files is constrained only by the compiler and the operating environment.	The number of open files is constrained only by the compiler and the operating environment.
Length of Field Names	10 characters Tables contained in a Visual FoxPro "database container" may have field names up to 128 characters in length. In such cases, there is also a "physical" name for each field in the table which has a maximum of 10 characters. That physical name is what will be visible to CodeBase Tools.	10 characters

Figure 31

NOTE(*): A reminder that tables in Large Mode can be read by Visual FoxPro or other 32-bit commercial applications provided they stay within the limits for compatibility given in the second column. However, if even one feature of the table exceeds its associated limit, the table will be unreadable by VFP, and data corruption could result.

History of Python CodeBase Tools

Back in 2008 M-P Systems Services, Inc. (MPSS), was faced with migrating away from the by-then discontinued Visual FoxPro (VFP) in a large on-line SaaS application to a different, modern programming platform. After exploring numerous options, including Java and .NET, the firm adopted Python as its strategic programming tool (at that time Python 2.6). The challenge was how to build Python applications which needed to share data contained in DBF tables created and updated continuously by the Visual FoxPro application modules. After much testing, the CodeBase C DLL, sold by Sequiter Software, was chosen over other options primarily for its compatibility and speed. VFP was known for its blazing fast performance in manipulating data, and CodeBase proved its performance equal for many essential processes like imports, exports, seeks, reindexing, and updates.

Python CodeBase Tools was developed by MPSS with two ideas in mind: 1st, to simplify the use of CodeBase for data handling by modelling the capabilities of the familiar (and very powerful) Visual FoxPro command structure, and 2nd, to exploit the speed and convenience of a C-language module using the Python C API to produce an importable Python component which didn't require ctypes or other intermediate Python components. What resulted is a C program with over 12,000 lines of code, which exploits the "Limited Python API" (and is compiled with Visual Studio to a .PYD file), which is version-neutral going forward, making it usable for Python versions 3.10 and up for the foreseeable future. The first version of Python CodeBase Tools was released in 2008 and relied on a C DLL loaded by Python using ctypes. The most recent update, implementing the "Limited Python API" was released for both 32-bit and 64-bit Windows Python, version 3.x in 2026. (A version-specific download is available also for 32-bit Windows Python 2.7, but no further development of the 2.7 component is planned. Version-specific modules are also available for 32-bit and 64-bit Python versions 3.7, 3.8, and 3.9. There are no plans for further updates of those older, now obsolete, versions. See the Python-CodeBase-Tools repository for more details.)

Before creating the Code-Base-Tools module in the Python Package Index (PyPI), a GitHub repository was created for both the Open Source version of Sequiter CodeBase and for this Python CodeBase Tools. These repositories are currently found at:

- https://github.com/MPSystemsServices/CodeBase-for-DBF - Contains all of the Sequiter CodeBase development files, the resulting DLLs compiled for Windows, and extensive documentation as originally supplied by Sequiter. Sequiter provided files which had once been used to compile Linux versions of the library, but these have not been tested or deployed.
- https://github.com/MPSystemsServices/Python-CodeBase-Tools - Contains all C source code for current and prior versions of C wrappers for the Windows C DLLs both 32-bit and 64-bit. Users wishing to customize or make contributions to this project are urged to clone this repository locally and explore all its documentation and features.